



赣东学院
GANDONG UNIVERSITY

本科生毕业设计（论文）

论文题目：_____基于 FreeRTOS 的智能居家安防系统_____

姓 名：_____吴兆国_____

学 号：_____20222042236_____

专 业：_____自动化_____

指导教师：_____段霖_____

完成时间：_____2026 年 5 月 10 日_____

基于 FreeRTOS 的智能居家安防系统

吴兆国

Smart Home Security System Based on FreeRTOS

ZHAOGUO WU

作者声明

我以诚信承诺如下：所提交的学位毕业设计（论文），是我自己在指导老师指导下独立完成的，不存在弄虚作假、抄袭剽窃等违反道德、学术规范的行为。文中引用他人文献、数据、图件、资料等均注明来源，未包含他人研究结果及为取得我校或其他教育机构学位或学历证书所使用材料。本毕业设计（论文）所引起的一切后果均由我个人承担。

本毕业设计（论文）成果归赣东学院所有。

特此声明。

毕业设计（论文）作者（签字）：

签字日期： 年 月 日

本人声明：这篇论文是我指导研究生所做研究工作结果，在论文完成后我认真检查了全文内容并确认题名、关键词、摘要等部分中英文一致准确。

学位论文指导教师签名：

签字日期： 年 月 日

摘 要

居民对于厨房火灾、燃气泄露以及门前有陌生人接近等风险越来越重视，但是很多家庭安全设备仍然局限于单一监测、仅在本地进行声音或光线提醒并且不能及时上传至云端，在报警时如果只是定时从队列中取出一个字符串进行判断，就有可能将状态的变化“推迟”到下一时刻才被发现。本设计基于 STM32F407VET6 实现了一个可以长时间插电工作的智能家居安防系统：LM393 火焰、MQ-2 烟雾、HC-SR04 超声波以及 DHT11 温湿度传感器通过 USART、SPI、I2C 等方式连接到 MCU，OLED 显示屏、蜂鸣器及多个 LED 用于现场显示和报警，MFRC522 用于刷卡开门，蓝牙以及 ASRPro 语音连接手机和个人口令两种人机交互方式，ESP8266 利用 MQTT 连接到 OneNET，进行状态上传及接收来自云端开关等命令。软件方面使用 FreeRTOS，把采集、声光驱动、菜单与存档、模组收发拆为多个任务合作完成：上电后先运行一个仅执行一次的任务进行初始化工作，在创建必要的内核对象和外设之后退出，然后一直运行的任务中使用互斥量保护串口打印、有效卡号以及火/烟报警量等共享数据，通过消息队列传递 OLED 绘制、EEPROM 读写、蜂鸣器与 LED 操作命令以及 ESP8266 一帧回送，二值信号量用于火焰、烟雾模块的 DO 引脚电平变化通知蓝牙与语音任务读取本地缓存，事件标志组用于按键与 RTC 唤醒。

关键词： 智能安防； FreeRTOS； 嵌入式系统； 多传感器融合； 物联网

ABSTRACT

There is an increased household focus on fire in the kitchen, unusual gas situations, and persons near the entrance, but most of the home security solutions still follow an outdated scheme, namely single-channel detection, typically sound and light, and a slow update at the remote end, and even if the alarm message is polled out of a queue it may still take several polling cycles to receive a state change. The design uses a smart-home security prototype that is constantly powered by STM32F407VET6: flame detection (LM393) and smoke detection (MQ-2), ultrasonic ranging (HC-SR04) and temperature (DHT11) are connected to the MCU using USART, SPI, I2C and other interfaces; on-site display and prompts are provided by OLED, a buzzer and multiple LEDs; MFRC522 manages swipe-card access; Bluetooth and an ASRPRO voice module are used to communicate with both phone-side and spoken human-machine communication; an ESP8266 connects to OneNET over MQTT to integrate property reporting and cloud-side on-off type commands. FreeRTOS is selected on the software level to divide the sampling process, actuator drive, menu and archive service, and module traffic into collaborating tasks: upon power-on, a one-shot init task will setup RTOS objects and peripherals and then delete itself, mutex variables will shield common data, like printf, the correct card ID and numbers of fire/smoke alarms, queues will forward OLED drawing, EEPROM read/write, buzzer and LED actions bytes, and one ESP8266 return frame, binary semaphores will signal the Bluetooth and voice tasks to read the on-board cache whenever the DO pins on the flame and smoke modules switch, and the event group will combine keypad events with RTC wakeups.

Key Words: Smart Security; FreeRTOS; Embedded System; Multi-sensor Fusion; Internet of Things;

目 录

绪论..... 1

1.1 研究背景..... 1

1.2 研究意义..... 1

1.3 国内外研究现状..... 1

1.3.1 国外研究现状..... 1

1.3.2 国内研究现状..... 2

1.4 研究内容与论文结构..... 2

1.4.1 本文主要研究内容..... 2

1.4.2 论文结构安排..... 3

第二章 系统需求分析与总体方案设计..... 4

2.1 系统需求分析..... 4

2.2 总体方案论证与比选..... 5

2.3 总体技术路线与逻辑分层..... 5

2.4 本章总结..... 6

第三章 硬件选型与电路设计..... 7

3.1 硬件模块选型..... 7

3.1.1 主控制器选型..... 7

3.1.2 传感器信息采集模块选型..... 7

3.1.3 人机交互与身份认证模块选型..... 8

3.1.4 远程通信模块选型..... 8

3.1.5 持久化存储模块选型..... 8

3.2 主控制器电路设计..... 8

3.3 传感器信息采集电路设计..... 9

3.3.1 温湿度模块电路设计..... 9

3.3.2 烟雾传感器模块电路设计..... 10

3.3.3 火焰传感器模块电路设计..... 10

3.3.4 超声波测距模块电路设计..... 10

3.4 人机交互与身份认证电路设计..... 11

3.4.1 RFID 射频识别模块电路设计..... 11

3.4.2 OLED 显示模块电路设计..... 11

3.4.3 语音识别模块电路设计..... 12

3.5 远程通信电路设计..... 13

3.5.1 蓝牙通信模块电路设计..... 13

3.5.2 WiFi 通信模块电路设计	13
3.6 持久化存储模块电路设计	14
3.7 本章总结	14
第四章 系统软件设计	15
4.1 FreeRTOS 多任务总体设计	15
4.2 系统初始化任务设计	17
4.3 传感器数据采集任务设计	18
4.3.1 温湿度采集任务	18
4.3.2 烟雾检测任务	18
4.3.3 火焰检测任务	19
4.3.4 超声波测距任务	20
4.4 人机交互任务设计	20
4.4.1 RFID 门禁任务设计	21
4.4.2 OLED 显示与按键菜单任务设计	22
4.4.3 语音识别任务设计	23
4.5 远程通信与云平台任务设计	24
4.5.1 蓝牙通信任务设计	24
4.5.2 WiFi 与 MQTT 云平台任务设计	26
4.6 EEPROM 持久化存储任务设计	27
4.7 报警机制与 FreeRTOS 内核组件设计	27
4.8 本章总结	28
第五章 系统调试与测试	29
5.1 硬件调试	29
5.2 传感器功能测试	30
5.3 通信模块测试	31
5.4 FreeRTOS 多任务运行测试	34
5.5 系统整体联调测试	35
5.6 本章总结	36
结 论	37
参考文献	38
致 谢	39
附 录	40

绪论

1.1 研究背景

厨房明火、燃气泄漏以及门口异常接近，在家庭环境中发生频率较高，也最容易被房主察觉。如果仅做一路检测再加上本地声光报警，在有人的情况下一般还是事后来电；而手机端如果不及时刷新也会造成滞后。所以更加合理的需求是把这些多种传感器的结果整合起来做出判断，并且让本地提醒跟远程监控连贯起来，而不能仅仅做到“响”。

板子外设多、接口时序不同，如果全部采用裸机大循环等待，某一处阻塞就会影响整个程序读取数值。而 FreeRTOS 程序体积较小且可以裁剪，在本课程设计所使用的单片机平台上使用更为合理：将传感器采集、OLED 显示、串口通信以及报警功能划分为多个任务之后，在各个任务间通过消息队列或者信号量进行通信及互斥，调试更加方便，在撰写毕业设计报告时也便于将各个部分的功能分块描述。

1.2 研究意义

本课题将 FreeRTOS 应用于智能家居安防样机开发，以可复核工程为例，在此基础上实现多个传感器读取以及 SPI/I2C/USART 多种通信方式接入，同时将本地报警、门禁管理以及 MQTT 上云集成到一个任务当中进行。此项目可以供相同嵌入式安防课程设计人员参考，同时也丰富 FreeRTOS 在智能家居领域的应用实例。

对于本课题的设计与调试过程中，单片机原理、传感器的应用、嵌入式操作系统以及通信技术均在一个系统上进行了联合测试，在毕业设计实践中所起到的作用更多是硬件电路的设计、任务级软件编程以及多个模块配合的能力，而非单纯的某一方面的功能简单叠加。

1.3 国内外研究现状

1.3.1 国外研究现状

国外对于智能家居安防的研究起步较早，已有一套完整的技术方案到产品的流程。目前大多数相关研究集中在嵌入式实时操作系统的应用上，比如多传感器协同、人机交互等，FreeRTOS、 $\mu\text{C}/\text{OS-II}$ 等轻量级的操作系统在安防设备上的多任务调度得到了比较广泛的应用^[1]。

霍尼韦尔、博世、松下等公司在国际市场上的家居安防产品，一般都将红外线、烟雾、天然气等多种传感器连接至一个中心节点，在现场进行检测后通过 MQTT 或 CoAP 等协议传输到手机以及云端，这一点在相关文献中提及较多。而对本文更有针对性的内容却是围绕嵌入式的底层技术展开的，如任务调度能否适应事件的发生频率、传感链路如何防止错误判断、无线通信在网络较差的情况下仍能正常工作等，在阅读这些内容时可以将其视为一种参考标准而非直接应用于产品的标准^[2]。

1.3.2 国内研究现状

国内的研究起步较晚,但是在教学验证以及工程化应用上发展迅速。目前的研究主要集中在基于单片机安防系统、多种传感器综合监控、远程控制以及嵌入式操作系统等方面的应用上,在近年来的文章当中对于使用 FreeRTOS 开发家庭安防设备的相关报道较多^[3]。

王利强等^[4]在 STM32 上基于 FreeRTOS 建立一个多任务系统并且进行了实现,在文章中对于信号量以及事件标志组如何用于共享资源保护进行了详细介绍。而在多传感方面,王宏等^[5]将火焰、烟雾及人体检测统一考虑连接方式及方法进行介绍。另外无线以及平台部分,则由朱兆芳等^[6]使用 ESP8266 通过 MQTT 连接设备与云端。三篇论文与本文第四章“先理顺内核对象,再分任务写出各个传感器链路,最后落在 ESP8266 以及 MQTT 上报”大致相同,在文章中可以逐个模块对比参考,但是不宜将文献结论当做自己产品性能保证。

同时,小米、海康威视、萤石等企业在消费级产品上已实现摄像头、烟感、门窗传感器等多种设备之间的连接以及手机端远程操控。这些已经实现的产品化案例是我们对于本课题功能边界的参考,在日常生活中我也使用过类似小米的产品,有很好的体验,因此我认为具有很高的借鉴意义。

1.4 研究内容与论文结构

如前所述,国外嵌入式实时操作系统以及安防整机工程化开展较早,霍尼韦尔、博世等企业已经将多传感、低时延联网及云端协议栈做成成熟商品化的产品,而学界更多关注调度相关的问题、通信鲁棒性等方面的内容^{[1][2]}。国内起步较晚,以 STM32 等为基础开发 FreeRTOS 以及多源感知、MQTT 上云的相关研究较多,而像小米、海康这类企业更多的是将摄像头、烟感、门窗磁等单品进行连接实现。但是关于“单片机上实现门禁、声光、蓝牙、语音、WiFi 上云及报警”的完整解决方案,目前可参考的较少^{[3][4]}。对比而言,国外强在于标准化产品以及长期维护,国内强在于更新速度快并且贴近生活,而本课题所选取的道路更偏向后者:在我的有限硬件条件基础上,将实时操作系统、多任务协同以及物联网上报等功能整合在一个 MCU 上完成整个项目的开发工作。

1.4.1 本文主要研究内容

产业侧将嵌入式操作系统推向具体的安防场景中应用,将视觉、语音等智能化技术融入到预警流程中,将模块做到尽可能的小、省电,这是未来几年的发展趋势^[7],本设计并非要实现一个功能完备的算法库,而是选取符合课程设计需求的部分进行开发:基于 STM32F407VET6 作为硬件平台,在 FreeRTOS 上读取火焰、烟雾、超声波、温湿度等环境量,利用互斥锁、消息队列、二值信号量、事件标志组将 OLED 菜单、蜂鸣器以及 LED、RFID 门禁、蓝牙以及 ASRPRO 语音、ESP8266 通过 MQTT 连接到 OneNET 这几条线分开后再连接起来,主要解决火、烟发生后的“谁需要立即获知、如何快速获

取信息而不等待字符串”的及时性问题，并在 EEPROM 中保存有效的身份证号以及超声波报警的距离等参数以便在现场调整。整机目标是在实验室环境下长期通电工作、外设互不影响抢占总线权利、出现故障能发出声音或者灯光并且远程同步报警，在此基础上为进一步增加摄像头或者更强的边缘计算能力预留总线及任务优先级资源。

1.4.2 论文结构安排

全文分为几大部分：第一章是引言部分介绍背景、目的以及国内外的研究情况，在 1.4 节总结了本文所做的工作以及各部分内容的安排；第二章首先根据“谁来使用、要解决的问题是什么”的原则划分客户侧的功能需求和非功能需求，然后提出总体的技术路线，在对比裸机轮询、RTOS 多任务、双 MCU 分工方案之后确定本设计所采用的分层方式，最后总结本章内容；第三章也是在本章中介绍了各个硬件模块的选择依据及其主要参数值，在此基础上给出了各个模块的工作原理及相互之间的连接关系，最后对本章进行小结。第四章在 FreeRTOS 框架下从各个任务的角度阐述初始化、门禁、传感采集、人机交互、远程以及云上通信、持久化存储以及报警内核对象等内容，在最后给出本章小结；第五章对硬件及其所连接的各种传感器、通信方式、多任务调度以及整个系统进行调试，方便读者（评审老师）查阅每部分内容。

2 系统需求分析与总体方案设计

2.1 系统需求分析

可以将客户看作提出委托的一方：住户或房东希望使用一块成本可控的板子，在厨房、门口等地方把烟、火、来人距离以及室内外温湿度监测，既要本地一眼明白，又要能够在手机或者云端心中有数；作为承接这个工作的毕设实现者，则要将这些模糊的想法分解为具体的可以检查的要求然后落实到硬件和任务分配中去。

从身份以及使用秩序上来说，客户不希望陌生人一开机就可以随意更改参数、查看菜单等，所以需要设备有如下功能：先证明我是谁，然后才能解锁进入操作的启动方式，在上电时只允许门禁读卡以及必要的后台逻辑，刷卡验证成功之后才显示时间界面、按键菜单、蓝牙以及语音等通道。而在工程实现上就是初始化时暂停 RTC、按键、超声波、蓝牙、语音等功能，在有效的卡片进行刷卡验证之后再恢复工作，形成硬性安全启动的要求。

环境以及安防相关问题，客户关注问题是：是否有烟、是否有明火、门口是否过近、室内的温度湿度大概是什么程度，而不是某种具体的算法名称；所以系统需要定时提供准确数值，在有情况发生时发出明显的报警声音或者闪光灯。火焰、烟雾最好使用安装简便、可调节阈值的传感器；门口的距离要与住户自己设置的安全距离进行对比，温度湿度只是用于舒适度以及联动使用。从功能上看，客户希望能够保存断电前的报警距离、权限卡号等信息并且可以在现场通过手机或者是维护端来修改而不是每当下电之后都需要重新下载。

报警以及通知及时性，在客户看来对于刚冒烟/刚起火状态下，手机或者语音多久知晓问题非常关心，所以在验收标准中需要有状态一旦改变即刻通知，而不能仅仅是一秒更新一次页面。近场侧除了蜂鸣器、LED 之外还需要有蓝牙推送以及语音播报功能；远端侧希望能够接入家庭 WiFi 的情况下，能够将重要参数及告警状态发送到常用的 IoT 平台，并且能够下发简单的灯光控制等操作以方便展示以及后续开发使用。

人机以及可维护性方面，在地应有显示屏及菜单以便不用看手机就能知道每个通道的情况；而在调试时客户（或指导老师）希望串口日志可读、整机能长期通电而不死机，这是从需求角度出发对于可靠性和可观测性的要求，而非消费级产品所必须满足的所有认证条件。

以上内容在本次毕设中实现方式，在源代码中任务分配及 IPC 设计一样：如温度湿度由单独采集任务入队供给菜单、蓝牙以及 MQTT 消费，火灾烟雾在 DO 边沿改变全局变量并以二值信号量唤醒蓝牙、语音任务，OLED 由单一任务消费显示队列，ESP8266 负责 MQTT 上云、下载 JSON 解析灯控等操作；具体函数及名称见第四章，此部分仅列出需求到能力的关系。

2.2 总体方案论证与比选

在单 MCU 情况下, 本设计曾尝试过三种总体方案。一种是裸机前后台: 主循环加上定时中断刷新各个外设, 好处是一目了然, 占空间少; 但坏处是超声波测距、ESP8266 长时间 AT 会话、DHT11 严格时序等较长临界区容易互相影响, 火烟状态如果只是用队列传输, 则蓝牙和语音提醒可能错过采样时间点, 不符合本文所提出的“状态变化即刻通知”。第二种是双 MCU 分工合作, 比如采集部分只负责传感工作, 而通信部分专门进行 WiFi 连接; 优点是职责明确, 但是缺点就是两个电源、两个调试接口以及两块电路板之间的协调增加了课程设计难度, 而且也不符合目前使用的单片机 F407 设计要求。第三种是单 MCU+FreeRTOS 多任务+队列/互斥/二值信号量/事件组, 各外设阻塞在 IPC 上, 由内核进行上下文切换填补空档, 报警链路与原代码相同。

到云的道路, 如果不用 MQTT 而是使用模块透传自定义短帧, 在手机端需要自己搭建服务器, 课堂演示闭合成本大; OneNET 端已有属性下发以及标准 MQTT 端口, 且与本文中 esp8266_mqtt.c 中 AT+MQTT 结合。从硬件成本、课程设计时间以及代码可验证性考虑, 采用 STM32F407VET6+FreeRTOS+分层架构+ESP8266 走 MQTT 上云作为本设计的整体方案。

2.3 总体技术路线与逻辑分层

总体仍以感知—控制—交互—云端四层来划分, 在第二章仅涉及边界以及数据流动情况, 单个器件引脚及电气细节在第三章阐述。

感知层负责将环境量转化为 MCU 可读电平或数字帧: 温湿度单总线、火烟 DO、超声回波时间、门禁 Mifare 卡号等; 控制层运行 FreeRTOS, 在初始化过程中创建所有的 IPC 对象以及业务任务, 在运行时通过调度器进行任务之间的切换; 报警相关的全局变量使用互斥来保证其一致性, 在状态发生变化时利用二值信号量唤醒蓝牙和语音任务。交互层包括 OLED 菜单、按键事件组、声光执行模块、蓝牙和语音串口等。云端层是由 ESP8266 保持连接, 在接收到计数停止后, 'app_task_monitor' 将一帧数据放入 'g_queue_esp8266' 中, 然后由 'app_task_esp8266' 解析下发灯控以及配合 'app_task_mqtt' 进行心跳和属性上报。

通电后如果仅绘制至模块级别而不写入函数名称, 则可以将数据流动视为三个步骤进行理解。首先是门禁: RFID 未认证之前, OLED 菜单和蓝牙、语音等功能处于休眠状态; 认证之后, 这些近距离连接功能得以唤醒, 本地显示以及串口通信也得以开放。其次是一般性监测与告警: 超声波、温度湿度等每隔一段时间采集一次数据, 火焰、烟雾在数字量发生变化时更新缓存并在报警路径中(包括队列入队及信号量通知); 如有异常情况发生, 就会有声音灯光以及蓝牙、语音警告一同发出。最后一个是上传云端: ESP8266 一直保持 MQTT 连接, 温度湿度以及各个开关量根据设定的时间间隔上传到云端。以上三步在第四章具体到相应的任务、队列以及信号量上, 这里不做进一步细化

到函数层面。

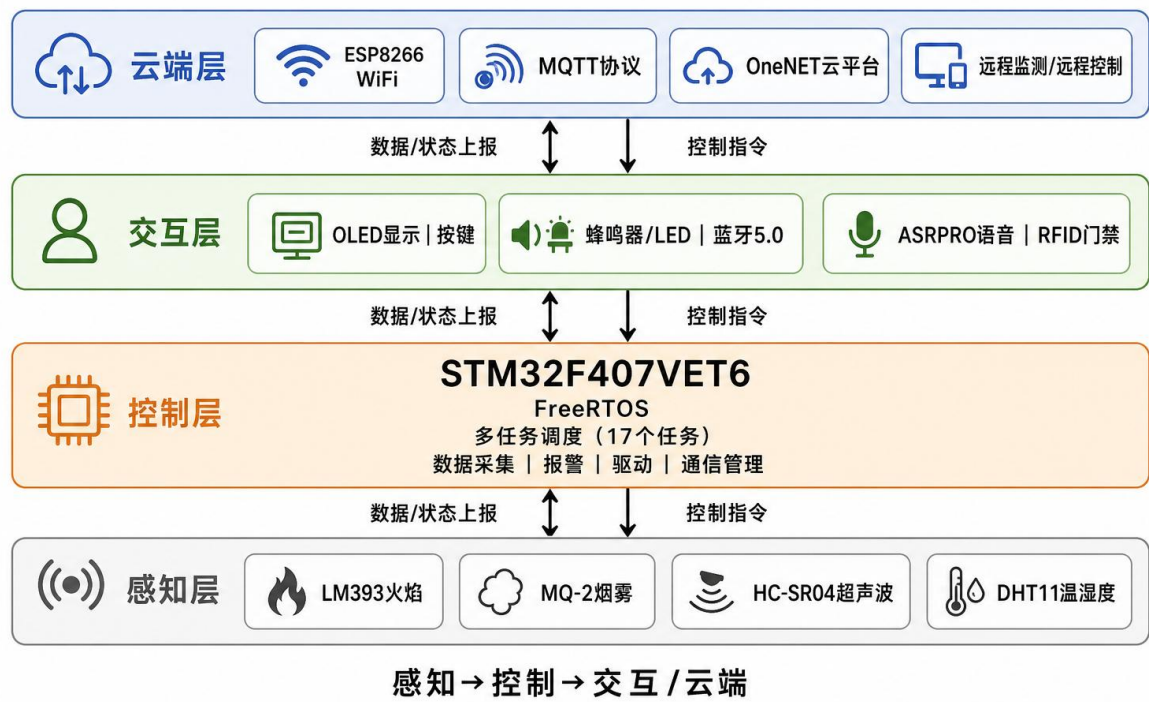


图 2-1 系统总体架构图

2.4 本章总结

这一节将使用方案分解为可验收条目：上电后需先完成门禁才可打开菜单并进入通道，环境量需包括烟、火、门口距离以及温湿度，报警需在状态发生变化之后立即向声光、蓝牙、语音以及云端发出，卡号以及报警距离这些内容都可以进行更改并且断电不会丢失还要有本地显示以及串口可见度，在单 MCU 限制条件下对比了裸机轮询、双 MCU 分工以及 FreeRTOS 多任务这几种方式，同时由于要连接到互联网所以介绍了采用 MQTT 以及 OneNET 进行演示。分层模型以及上电后粗数据只写到逻辑边界，在图 2-1 中将系统分为感知、控制、交互、云端四部分，方便后面提及各个模块时都能回到这一张图进行定位。这同时对第三章器件及接口的选择起到一定的限制作用，也为第四章任务分配以及 IPC 设计提供了一个统一的标准，避免了论述或开发过程中出现各自为政的情况。

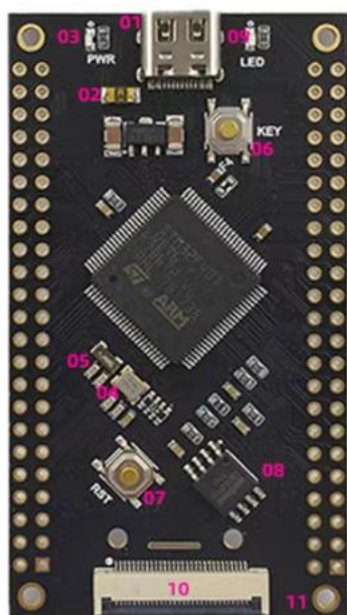
3 硬件选型与电路设计

3.1 硬件模块选型

本节沿用第二章确定方向，在此对各部分进行选择以及介绍其参数；引脚连接、上拉复位以及整个电路图详见下文各个电路小节中，避免重复。

3.1.1 主控制器选型

主控采用 STM32F407VET6（Cortex-M4，168MHz，FPU），片内 Flash512KB，SRAM192KB，多个 USART 以及丰富的 GPIO 资源，可以连接蓝牙、语音、ESP8266 等并驱动多个传感器。相比于 STM32F103 系列，主频以及外设速率更快，在相同 FreeRTOS 配置情况下有利于实现更多的串口通信以及 OLED 队列刷新，符合本文需求及栈大小设置要求^[8]。



- ① USB:5V供电/数据通讯
- ② 自恢复保险丝
- ③ 电源LED指示灯
- ④ 主时钟晶振: 8MHz
- ⑤ RTC晶振: 32.768KHz
- ⑥ 用户按键
- ⑦ 复位按键
- ⑧ SPI flash W25Q16
- ⑨ 用户LED
- ⑩ 2.8寸液晶接口
- ⑪ 2mm铜柱固定孔

图 3-1 STM32F407VET6 核心板实物图

3.1.2 传感器信息采集模块选型

温湿度使用 DHT11 数字单总线传感器，能够满足一般家庭对环境粗略的要求，方便易用并且与 `dht11_get_msg` 周期性读取方法相同^[9]。烟雾是使用 MQ-2 模块 DO 引脚，通过连接在板载比较器电位器设置门限值，与程序中的 `mq2_get_do` 判断低电平表示有危险一致^[10]。火焰是采用 LM393 比较器型红外火焰探测器 DO 引脚，连接方式也与 `fire_get_d0` 相同，起到与烟雾相反作用，从而可以更全面地监控家中起火状况^[11]。选用 HC-SR04，在 2~400cm 范围内由回声脉宽换算成距离，与 `sr04_get_distance()` 以及 `g_alarm_distance` 比较相同思路^[12]。

3.1.3 人机交互与身份认证模块选型

门禁使用 MFRC522，协议层面完善，符合本文所使用的软件 SPI 时序驱动方法^[13]。显示使用 0.96 英寸 128×64 OLED，本地信息量较少，通过 `g\_queue\_oled` 异步更新不会造成多个任务竞争总线问题^[14]。语音使用 ASRPRO 离线识别模块，串口输出字符串命令，方便在 FreeRTOS 下单独的任务中进行解析。

3.1.4 远程通信模块选型

蓝牙采用 5.0 透传模块，通过 USART4 与手机串口助手即可实现参数查询以及灯控^[15]。WiFi 使用 ESP8266，利用其自带 TCP/IP 及 AT 指令加快开发速度，接入 OneNET 及源代码中 `esp8266\_mqtt\_init` 相同^[6]。

3.1.5 持久化存储模块选型

采用 AT24C02（256B，I2C），其容量足以存储 5 字节卡号以及 1 字节报警距离；写寿命满足课程设计要求，在单独 EEPROM 任务中对其进行串行操作，如第四章所述。

3.2 主控制器电路设计

STM32F407VET6 最小系统电路包括该芯片、电源电路、复位电路、晶振电路以及 JTAG/SWD 调试接口。其中电源由 3.3V 电压提供并经过一个去耦电容进行滤波以保持电源稳定；复位电路是用一个 RC 复位电路并且有一个手动复位按钮；外部晶振使用的是 8MHz 有源晶振并通过内置 PLL 倍频到 168MHz 作为系统的主频；而调试接口则是使用了 SWD(2 线)方式并通过 PA13(SWDIO)和 PA14(SWCLK)与 J-Link 相连。

串口 1（PA9-TXD，PA10-RXD）使用 USB 转串口芯片连接到 PC 上，用来打印系统调试信息（printf 重定向到串口 1）。

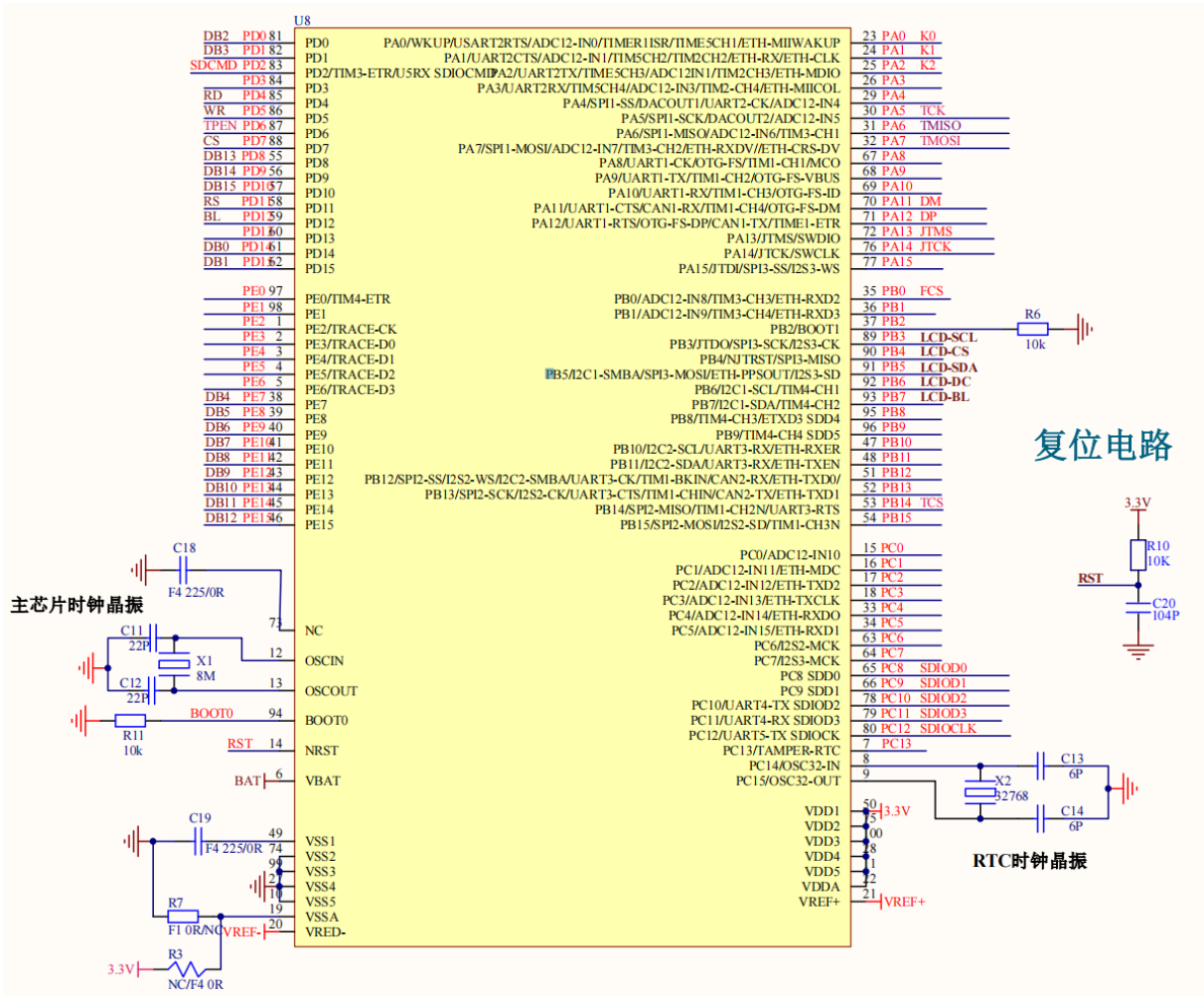


图 3-2 主控制器最小系统电路原理图

3.3 传感器信息采集电路设计

3.3.1 温湿度模块电路设计

DHT11 和 MCU 是通过单总线进行通信，数据线连接到 PE9。总线空闲时应为高电平，在 DATA 和 3.3V 之间连接一个 4.7kΩ 上拉电阻；读写操作时还是由传感器和 MCU 根据时序把线拉低或者释放。模块电源使用 3.3V,VCC 连接到 MCU VDD; GND 接到系统地。

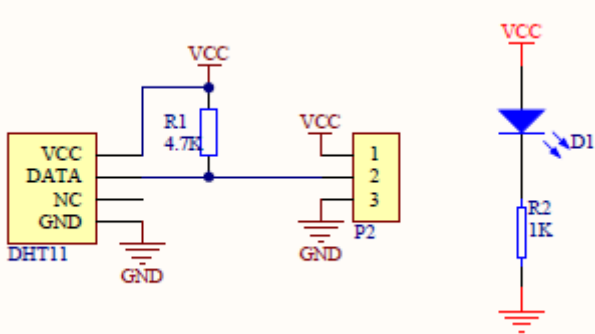


图 3-3 DHT11 温湿度模块电路原理图

3.3.2 烟雾传感器模块电路设计

MQ-2 烟雾传感器模块供电为 5V，GND 连接到系统地，数字输出引脚 DO 连接到 PA1（GPIO 输入模式）。该模块包含一个 LM393 比较器，在上面有一个可调电位器用于设置报警门限即灵敏度，当气体浓度超过一定值后，DO 引脚就会输出低电平，而主控就可以根据 PA1 电平的变化来检测烟雾情况。

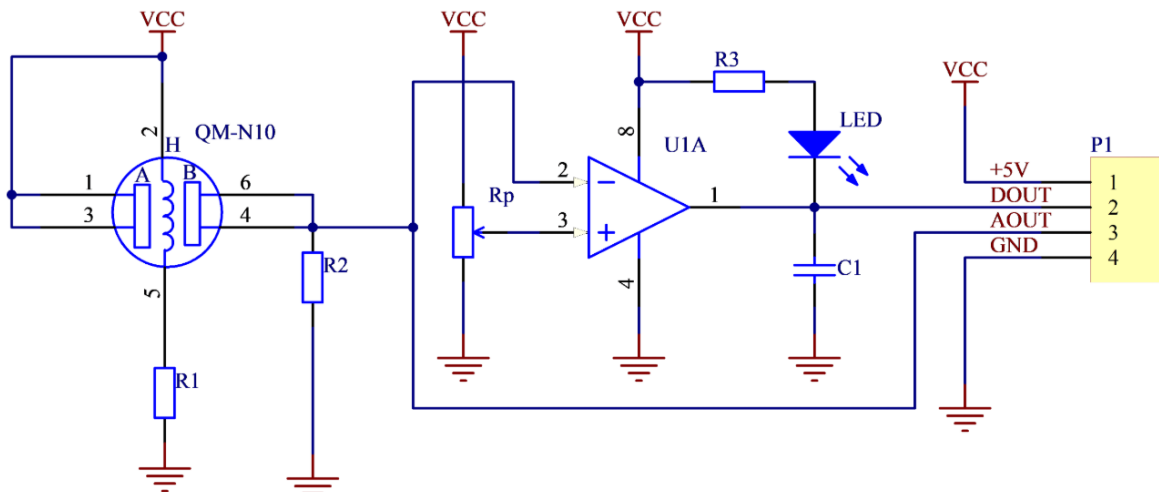


图 3-4 MQ-2 烟雾传感器模块电路原理图

3.3.3 火焰传感器模块电路设计

LM393 火焰传感器模块供电为 5V，GND 连接到电路的地，数字输出端 DO 连接到 PB7（GPIO 输入模式）。该模块集成一个 LM393 比较器，当检测到火焰发出的红外线后 DO 引脚就会拉低电平，微控制器可以通过读取 PB7 上的电平来得知是否有火焰存在。

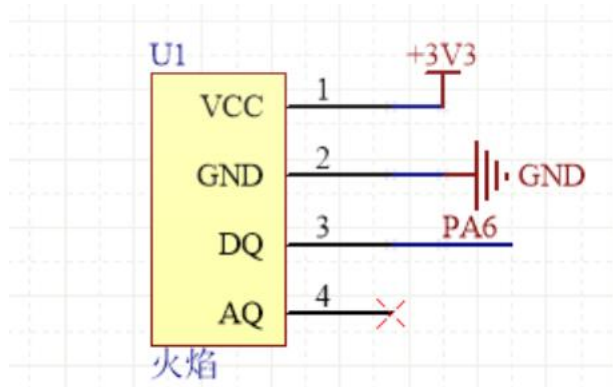


图 3-5 LM393 火焰传感器模块电路原理图

3.3.4 超声波测距模块电路设计

HC-SR04 超声波模块接 5V 电源，GND 连接到地；TRIG 连接到 PB6（GPIO 输出），ECHO 连接到 PE6（GPIO 输入）。模块工作时使用 PB6 发出一个至少 10us 宽度的高电平信号来触发一次测量，在 PE6 读取返回的高电平所占时间；然后用这个时间乘以常数得到距离：距离（cm）= 高电平时间（μs）× 声速（340m/s）/ 2 / 10000。

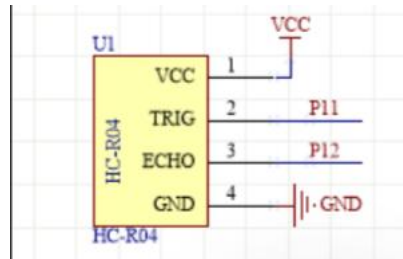


图 3-6 HC-SR04 超声波测距模块电路原理图

3.4 人机交互与身份认证电路设计

3.4.1 RFID 射频识别模块电路设计

MFRC522 RFID 模块供电为 3.3V，GND 连接到单片机的地，用 SPI 进行通信：CS 连接到 PD6，SCLK 连接到 PD7，MOSI 连接到 PC7，MISO 连接到 PC8，RST 连接到 PC9。SPI 通信使用软件模拟的方法，利用 GPIO 来产生 SPI 时序，从而读取或者写入 MFRC522 寄存器。

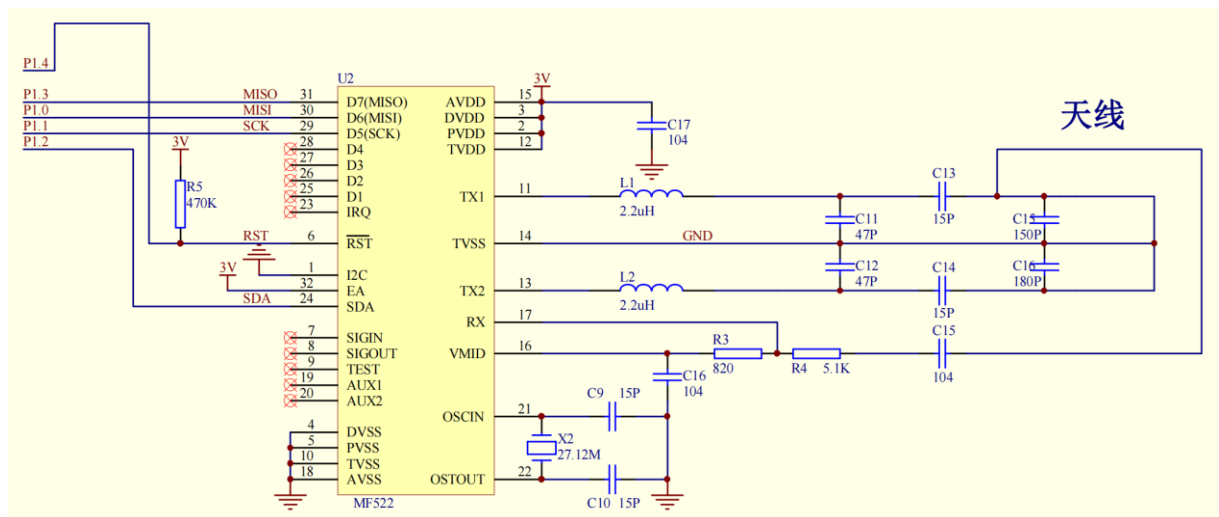


图 3-7 MFRC522 RFID 模块电路原理图

3.4.2 OLED 显示模块电路设计

OLED 显示屏供电是 3.3V（或者 5V），GND 连接到系统的地线，用软件仿真的方式进行 IIC 操作：SCL 连接到 PB15，SDA 连接到 PD10。所使用的驱动芯片是 SSD1306，在发送一个命令字节来设置屏幕参数之后再发送一个数据字节用来写入要显示的内容。而在这个项目中是使用消息队列(g_queue_oled)来异步控制 OLED，OLED 的任务被阻塞直到有消息到来才会去处理这个消息从而完成相应的显示工作，这样就可以保证多个任务可以同时使用 OLED 而不发生冲突。

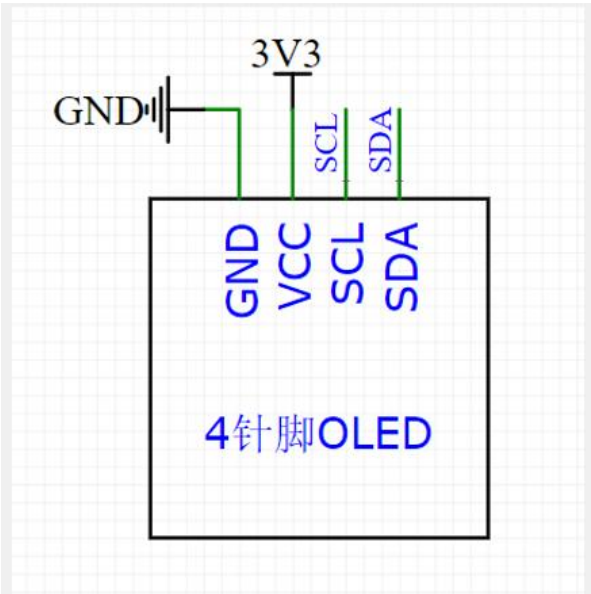


图 3-8 OLED 显示模块电路原理图

3.4.3 语音识别模块电路设计

ASRPRO 语音识别模块供电为 5V，GND 连接系统地，用 USART2 与主控 STM32 连接：PA2 连接 ASRPRO RXD，PA3 连接 ASRPRO TXD，波特率为 9600bps，该模块集成语音识别算法，具有离线识别能力，识别结果通过串口以字符串的形式发送给主控；同时主控也可以通过串口向模块发送播放命令（例如“1#”表示播放“灯已开”），即实现语音反馈。

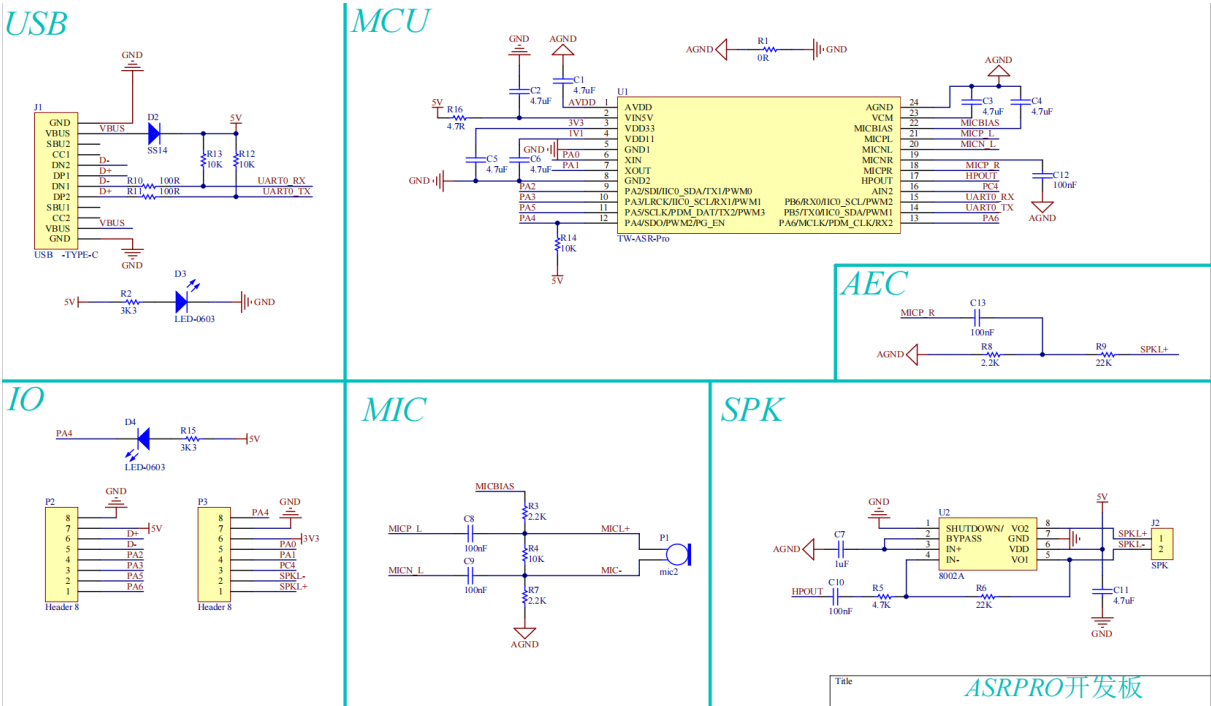


图 3-9 ASRPRO 语音识别模块电路原理图

3.5 远程通信电路设计

3.5.1 蓝牙通信模块电路设计

蓝牙 5.0 模块供电 5V，GND 接到系统地，用 USART4 与主控通信：PC10 连接蓝牙模块 RX，PC11 连接蓝牙模块 TX，波特率为 9600bps。蓝牙模块处于透传模式，在手机端用蓝牙串口助手软件连接到该模块之后就可以实现与主控间的数据交互。

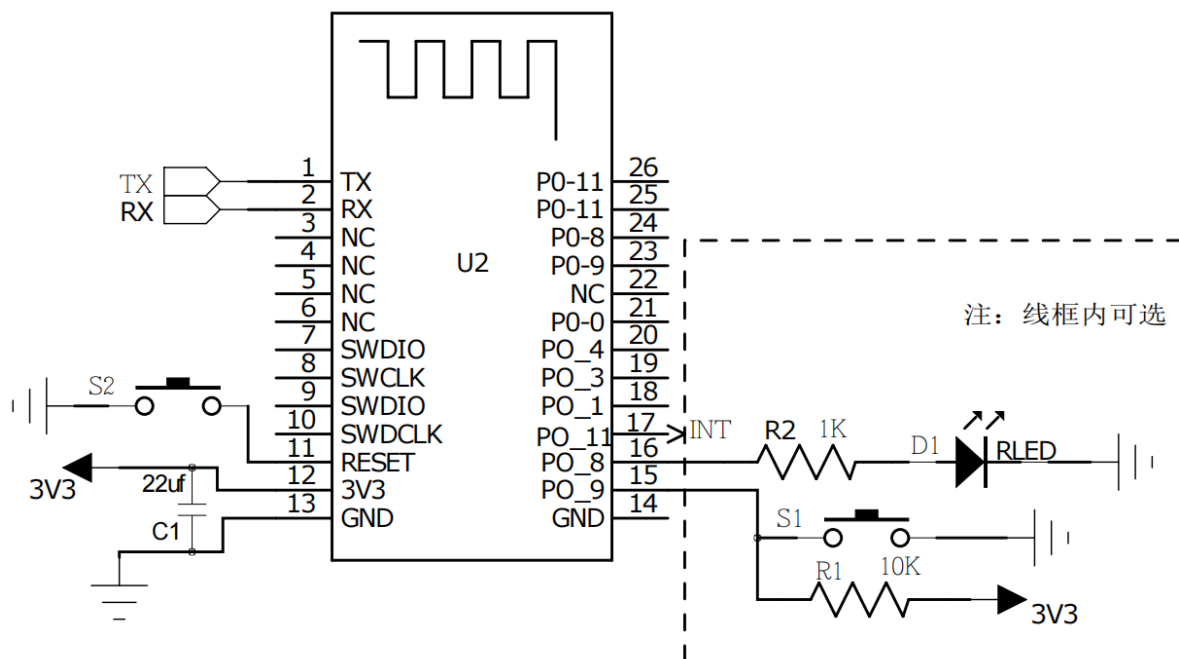


图 3-10 DX-BT24 蓝牙 5.0 通信模块电路原理图

3.5.2 WiFi 通信模块电路设计

ESP8266 WiFi 模块供电为 3.3V，GND 连接系统地，采用 USART3 进行通讯：PB10 连接 ESP8266RX，PB11 连接 ESP8266TX，波特率为 115200bps。模块使能引脚（EN）连接到 3.3V 以保证其处于开启状态，RST 引脚利用 RC 电路实现上电复位。主控利用 AT 指令控制 ESP8266 建立 WiFi 连接、TCP 连接以及 MQTT 通信等。

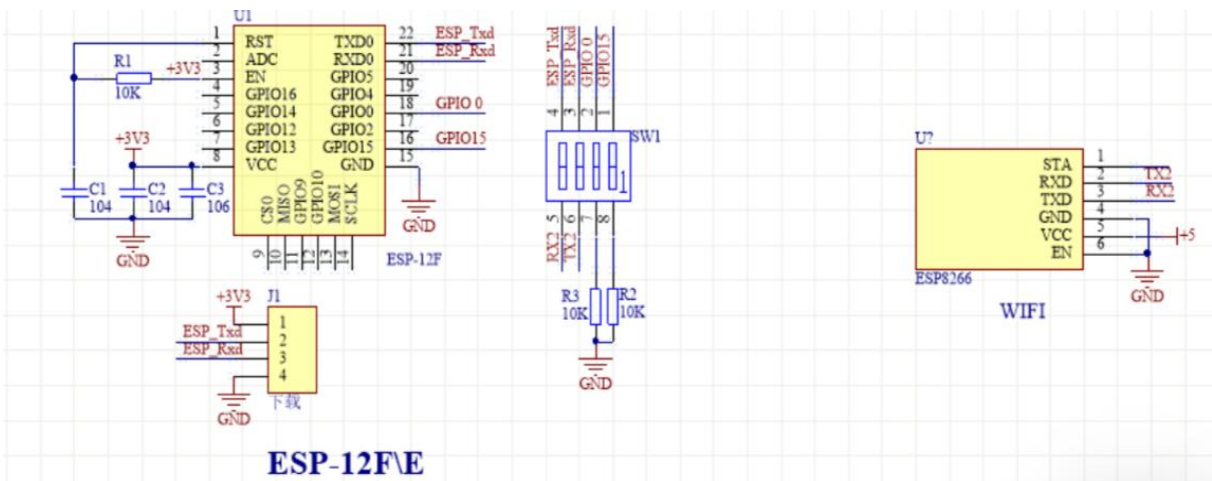


图 3-11 ESP8266 WiFi 通信模块电路原理图

3.6 持久化存储模块电路设计

AT24C02 EEPROM 芯片供电为 3.3V，GND 连接到系统地，用 I2C 方式与 MCU 进行通信：SCL 连接到 PB8，SDA 连接到 PB9，SCL 以及 SDA 都通过一个 4.7kΩ 上拉电阻连接到 3.3V 电源。芯片的地址引脚 A0、A1 和 A2 都接地，所以 I2C 设备地址是 0xA0（写）/0xA1（读）。

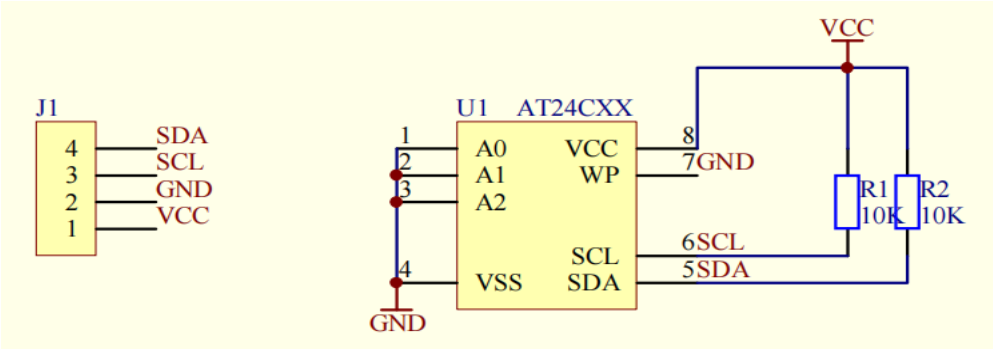


图 3-12 AT24C02 EEPROM 模块电路原理图

3.7 本章总结

本章将选型依据以及电路实现合并在一章：3.1 节根据主控、传感、人机、通信与存储五个方面介绍为何选择这些模块以及它们与软件任务的关系，在 3.2~3.6 各个原理图小节中介绍了主要引脚以及供电等信息，以减少选型部分和电路部分的简单重复。从图号来看，核心板实物为图 3-1，最小系统及其外围设备原理图依次编号至图 3-12，顺序与图号递增一致，在 Word 中可以方便地插入章节中间插图。通过以上安排，当读者在第四章阅读任务和队列相关的内容时，可以随时回来看一看本章中的硬件接口是否能够满足需求，从而使硬件章节和软件章节可以互相参考。对于毕业设计写作来说，本章既满足了课程设计对电路介绍部分的要求，也未影响到第二、四章的需求以及及时性。

4 系统软件设计

4.1 FreeRTOS 多任务总体设计

本系统基于 FreeRTOS 将其划分为 17 个任务，每个任务负责一个硬件设备或者一类业务操作，在这些任务之间通过消息队列、信号量以及事件标志组进行信息传递与互斥控制，使得采集、显示、告警、门禁、通信及上云等操作可以同时进行并且方便排查故障。

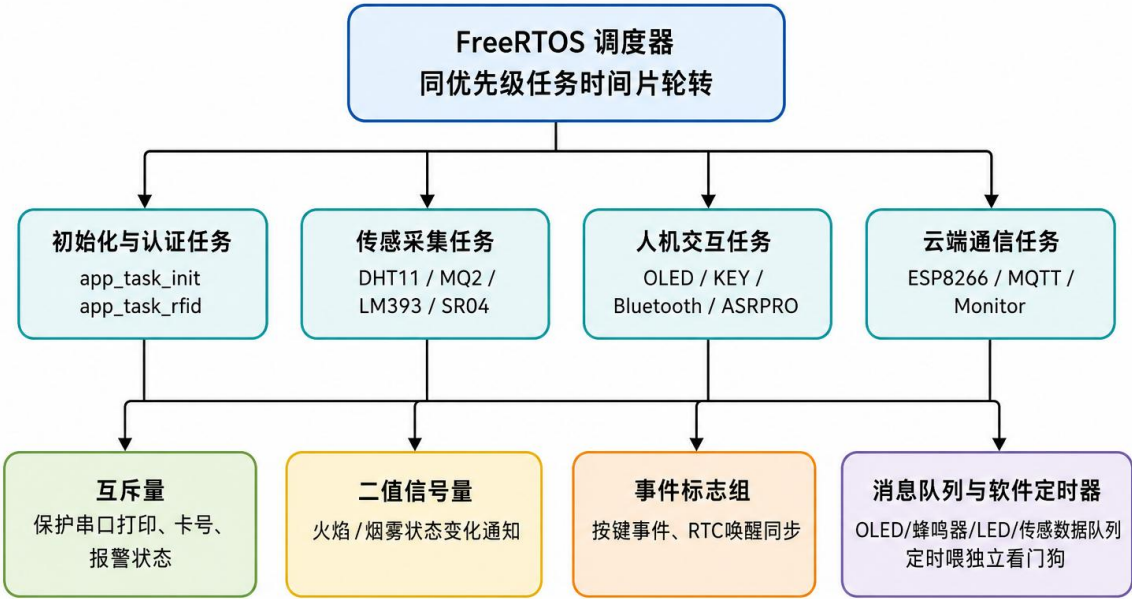
系统任务列表如表 4-1 所示。

表 4-1 系统 FreeRTOS 任务列表

任务名称	任务函数	栈大小（字）	优先级	功能描述
app_task_init	app_task_init	512	5	系统初始化，创建所有内核对象与子任务
app_task_oled	app_task_oled	512	5	OLED 显示驱动，消息队列异步刷新
app_task_rfid	app_task_rfid	512	5	RFID 门禁身份认证
app_task_eeprom	app_task_eeprom	512	5	EEPROM 读写管理
app_task_led	app_task_led	512	5	LED 指示灯控制
app_task_beep	app_task_beep	512	5	蜂鸣器报警控制
app_task_rtc	app_task_rtc	512	5	RTC 实时时钟读取与显示
app_task_key	app_task_key	512	5	按键扫描与菜单导航
app_task_sr04	app_task_sr04	512	5	超声波测距与入侵检测
app_task_lm393	app_task_lm393	512	5	火焰传感器检测与报警
app_task_mq2	app_task_mq2	512	5	烟雾传感器检测与报警
app_task_dht11	app_task_dht11	512	5	温湿度数据采集
app_task_bluetooth	app_task_bluetooth	512	5	蓝牙通信与指令解析
app_task_aspro	app_task_aspro	512	5	语音识别与语音播报
app_task_mqtt	app_task_mqtt	512	5	MQTT 数据上报
app_task_esp8266	app_task_esp8266	1536	5	ESP8266 WiFi 连接与云端通信
app_task_monitor	app_task_monitor	1024	5	ESP8266 接收数据监控

系统内核对象有三个互斥信号量（g_mutex_printf 保护串口打印，g_mutex_card 保

护 RFID 卡号, g_mutex_alarm 保护报警状态全局变量); 四个二值信号量(g_sem_fire_bt、g_sem_fire_asr、g_sem_mq2_bt、g_sem_mq2_asr, 用于火焰或烟雾发生变化时进行通知); 一个事件标志组 (g_event_group, 用于 RTC 唤醒以及按键事件同步)。8 个消息队列 (g_queue_oled,g_queue_eeprom,g_queue_led,g_queue_beep,g_queue_fire,g_queue_mq2,g_queue_dht11,g_queue_esp8266); 1 个软件定时器(周期为 1000ms, 每到此时间点就调用一个函数给独立看门狗喂狗)。



各任务通过FreeRTOS内核对象完成同步、通信与资源保护

图 4-1 FreeRTOS 任务调度与内核对象关系图

系统启动过程是：main()函数完成 NVIC 优先级分组（第 4 组）、SysTick 设置、串口 1 初始化之后，只创建一个 app_task_init 任务并启动 FreeRTOS;app_task_init 任务完成全部硬件初始化、内核对象创建、子任务创建后自动删除自身，程序开始运行。

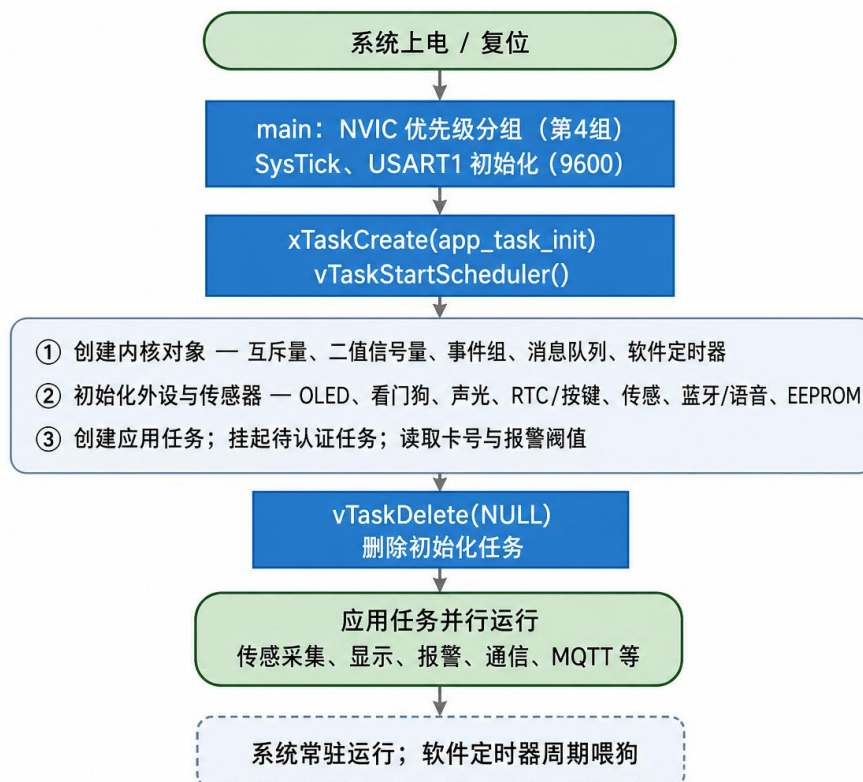


图 4-2 系统软件总体流程图

上图给出初始化及启动过程，在上电之后每个应用程序的任务都在 `for(;;)` 中循环执行并且被 FreeRTOS 调度器轮流执行。

4.2 系统初始化任务设计

`app_task_init` 是系统的启动任务，优先级为 5，栈大小 512 字节，它的工作内容有：

(1) 创建所有互斥信号量、二值信号量、事件标志组、消息队列；
(2) 创建一个软件定时器（周期为 1s），开启之后每隔一秒钟进行一次单独看门狗喂狗操作。

(3) 初始化所有硬件设备：OLED、独立看门狗（IWDG）、LED、蜂鸣器、RTC、按键、超声波、火焰传感器、烟雾传感器、温湿度传感器、蓝牙串口、语音识别串口、EEPROM 等。

(4) OLED 显示欢迎界面(锁屏图标以及“欢迎使用智能家居安防系统”字样)。

(5) 从 EEPROM 中读取 RFID 有效卡号(地址 0~4)：若是全部为 0xFF(第一次使用)，则写入默认卡号并更新到全局变量 `g_valid_card_id` 中；否则直接读取已保存值。

(6) 从 EEPROM 读取超声波报警距离（地址 5）：如果值是 0xFF(刚出厂)，写入默认距离 50 mm；否则读取该值到全局变量 `g_alarm_distance`。

(7) 按任务列表批量创建 17 个子任务；

(8) 挂起任务(`app_task_rtc`、`app_task_key`、`app_task_sr04`、`app_task_aspro`、

app_task_bluetooth)，待 RFID 认证成功后由 RFID 任务唤醒。

（9）自删除（vTaskDelete(NULL)）。初始化任务的流程图如图。

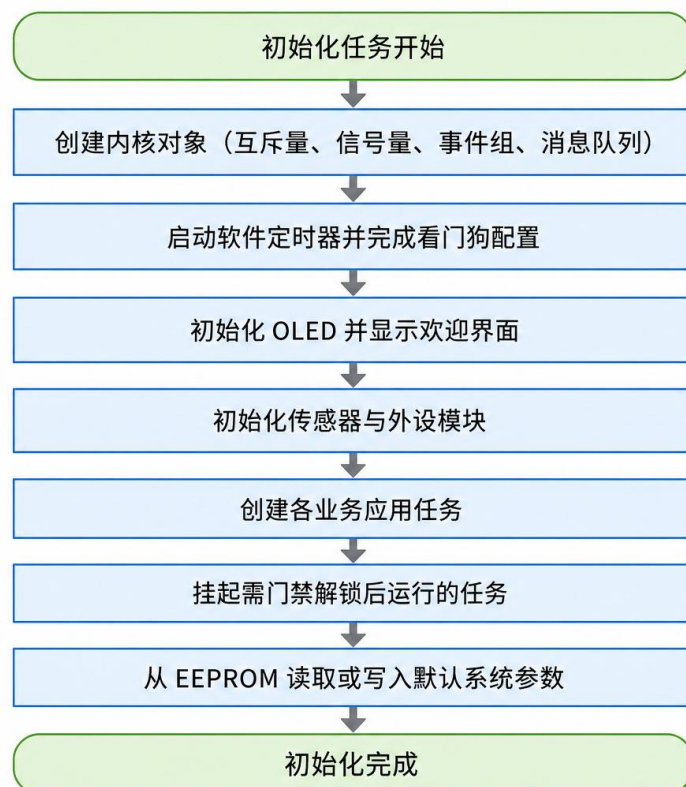


图 4-3 系统初始化任务流程图

4.3 传感器数据采集任务设计

4.3.1 温湿度采集任务

app_task_dht11 任务获取 DHT11 温湿度信息，每隔 6000 毫秒（DHT11 最小采样间隔时间）进行一次读取操作，在读取成功之后把五个字节的数据（湿度整数、湿度小数、温度整数、温度小数、校验值）放入到一个名为 g_queue_dht11 的消息队列中，由按键菜单任务、蓝牙任务、语音任务以及 MQTT 任务消费。

采集失败时通过串口 1 输出错误码，不上报队列，以免造成不良影响。全局变量 g_temp、g_humi 由 MQTT 任务从队列获取并赋值，用于上报到云端。

4.3.2 烟雾检测任务

app_task_mq2 用于监测 MQ-2 烟雾传感器的状态，在每 500ms 进行一次采样。该任务读取 PA1 引脚上的 DO 引脚电平（0 表示有烟即危险；1 表示无烟即安全），只当有变化时才做如下处理：

（1）获取该互斥锁 g_mutex_alarm，同步全局变量 g_mq2_status（0=安全，1=报警）。

（2）发送一个二进制信号量给 g_sem_mq2_bt（通知蓝牙任务），以及 g_sem_mq2_asr（通知语音任务）。

（3）报警时：给 `g_queue_beep` 发送蜂鸣器开命令，约 1000ms 后再发送关命令；同时给 `g_queue_mq2` 发送 "danger!!!"。

（4）恢复安全后：向 `g_queue_mq2` 发送 "safe"。

除了入队提示外，变换立即释放 `g_sem_mq2_bt` 和 `g_sem_mq2_asr`，蓝牙及语音任务可以以较短超时等待信号量获得最新的 `g_mq2_status`；而 `g_queue_mq2` 中的 "danger!!!" 和 "safe" 则用于需要显示相应内容一侧更新界面上的内容。相比于单纯依赖长时间周期性检查全局变量或者不断遍历队列中的陈旧信息，这种方法把提醒绑定在变换之后发生，可以避免被 500ms 的采样周期拉到队尾的感觉并且减少了不必要的查询次数，但是并不能做到整个流程的真正硬实时。

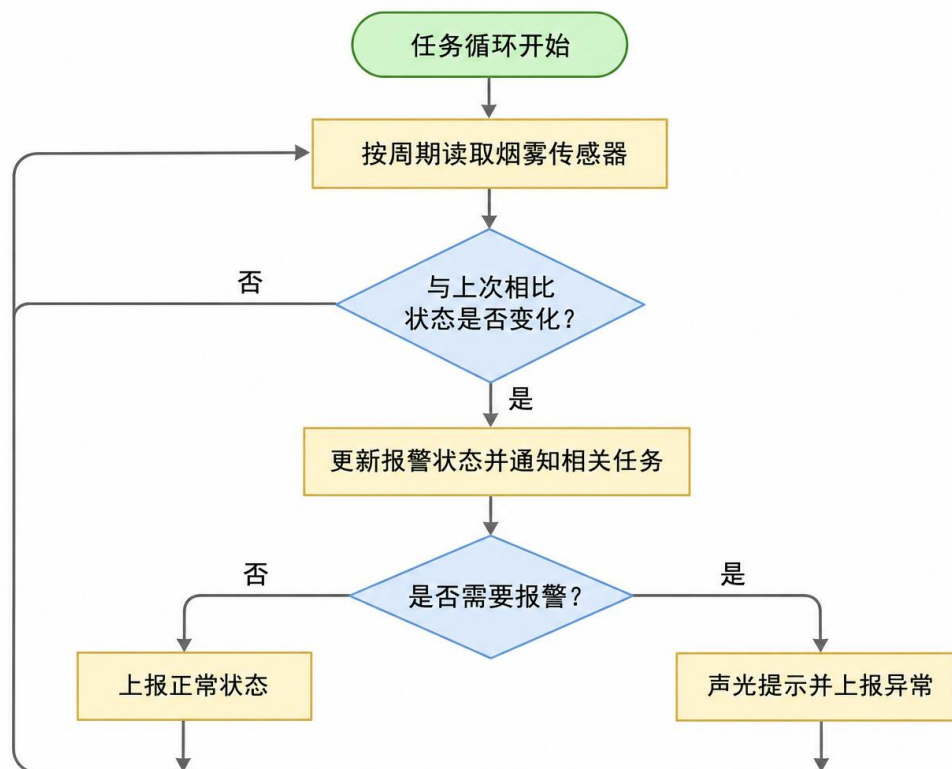


图 4-4 烟雾检测任务流程图

4.3.3 火焰检测任务

`app_task_lm393` 每隔 500ms 读取一次 LM393 模块 DO(PB7)：低电平表示检测到火焰、判危险，高电平表示未检测到火焰、判安全；该部分逻辑与 `app_task_mq2` 保持一致，在判断的结果相比前一帧有变化的情况下才进行下一步操作，其步骤为：

- （1）状态发生变化时获取互斥量 `g_mutex_alarm`，更新全局变量 `g_fire_status`；
- （2）发送二值信号量 `g_sem_fire_bt` 以及 `g_sem_fire_asr` 到蓝牙任务以及语音任务。
- （3）报警状态时：LED2 点亮并且蜂鸣器发出警报声并向 `g_queue_fire` 发送 "danger!!!"；
- （4）安全状态时：LED2 灭，蜂鸣器停止发声，给 `g_queue_fire` 发一条消息 "safe"。

火焰检测任务的流程图如下图所示：

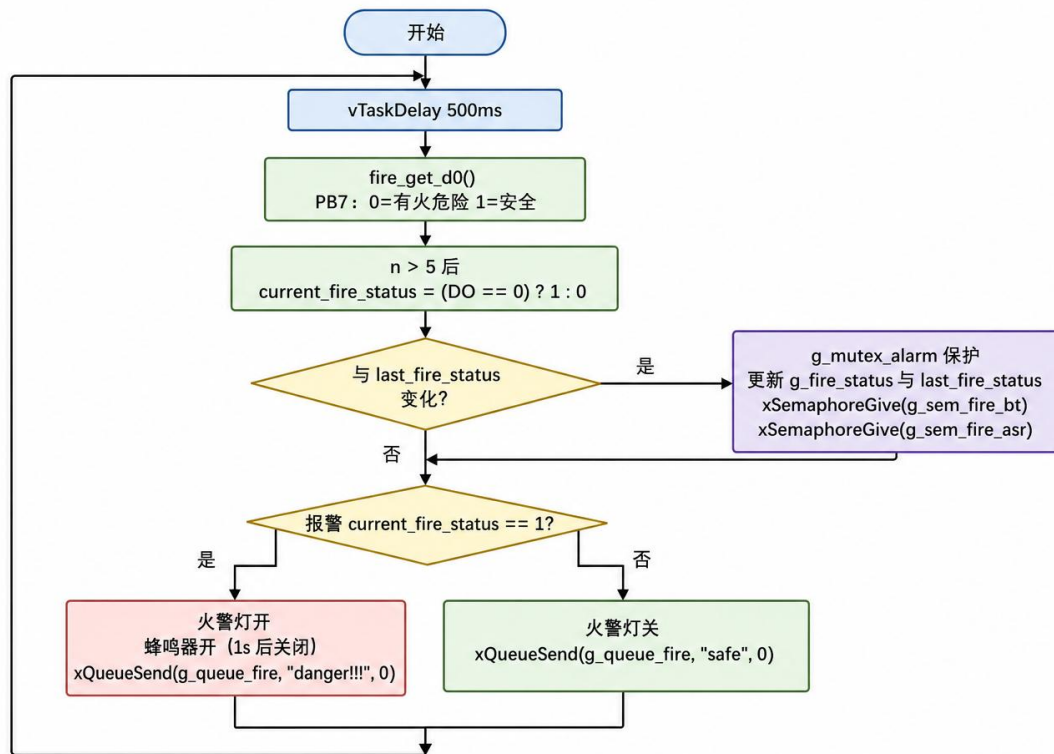


图 4-5 火焰检测任务流程图

4.3.4 超声波测距任务

app_task_sr04 每隔 500 ms 进行一次 HC-SR04 测量，用来检测门口是否有物体处于超声波报警区域内：PB6(TRIG)发出大约 10 μ s 的高电平来触发测量，在 PE6(ECHO)读取回波持续时间进而换算出距离；然后把得到的距离与 g_alarm_distance（上电后从 EEPROM 中获取的值）进行对比，其原则是

- (1) 距离小于报警阈值时：LED2 点亮，蜂鸣器响，OLED 上超声波图标报警。
- (2) 距离大于等于报警阈值时：LED2 关闭、蜂鸣器停止工作，在安全分支进行。

阈值默认为 50mm。蓝牙侧根据第二章指令表发送 DISTANCE SET-距离值之后，固件获取到新的阈值并保存至 g_alarm_distance 变量中然后写入 EEPROM 第 5 个位置上，下次开机也会读取这个位置上的值而不会让设置停留在 RAM 里。

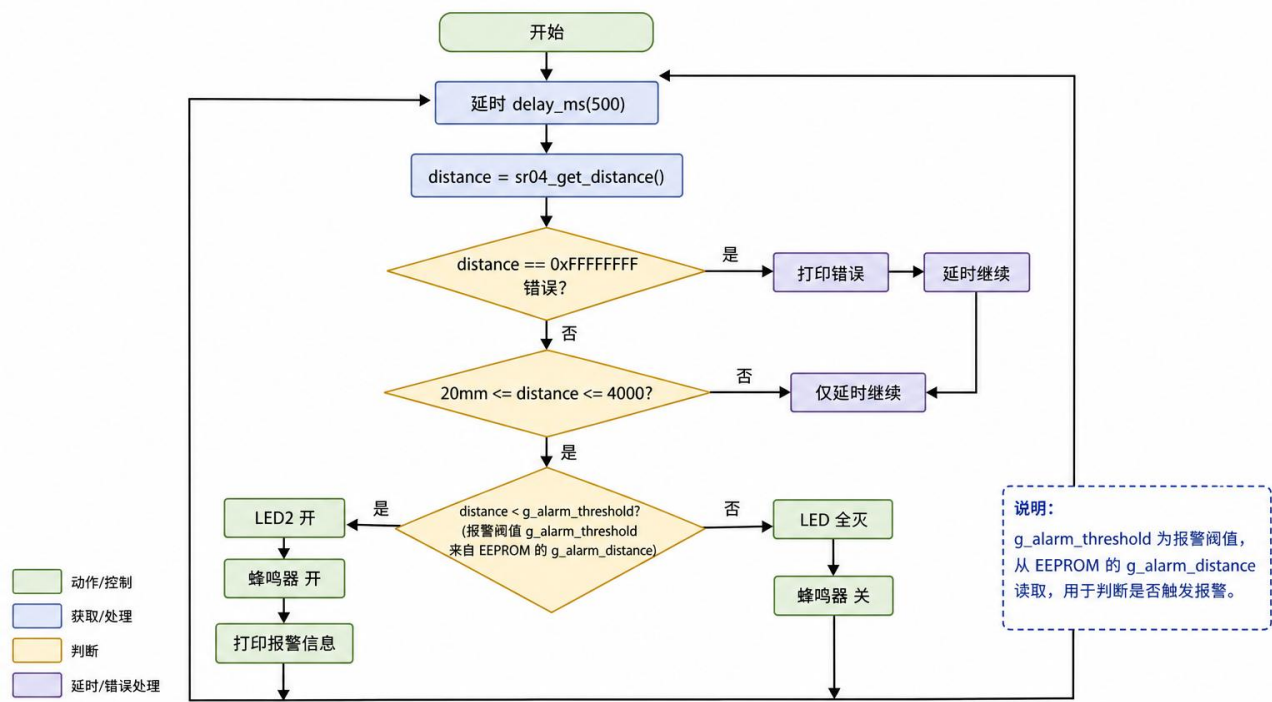


图 4-6 超声波测距任务流程图

4.4 人机交互任务设计

4.4.1 RFID 门禁任务设计

app_task_rfid 任务是系统门禁身份认证，也是系统安全启动第一道防线。该任务创建之后一直在运行，在 MFRC522 读卡器上不断读取 RFID 卡信息，并将其读取卡片编号与全局变量 g_valid_card_id 所保存有效卡号进行每一位比较。

有效卡认证流程：

- (1) MFRC522_Request 发送读卡请求，发现卡片之后进行防碰撞操作 (MFRC522_Anticoll)、选择该卡片 (MFRC522_SelectTag) 以及验证此卡片 (MFRC522_Auth)。
 - (2) 获取互斥锁 g_mutex_card，逐字节比较从文件中读取的卡号与 g_valid_card_id。
 - (3) 有效卡：LED0 闪一下，蜂鸣器响一下，OLED 显示解锁成功图标，然后唤醒 app_task_rtc、app_task_key、app_task_bluetooth、app_task_aspro 任务，自己被挂起（即 RFID 任务）。
 - (4) 无效卡：LED1 亮一下，蜂鸣器响两下，OLED 屏幕上显示错误标志，回到锁屏界面，继续等待刷卡。
- 有效卡号保存在 EEPROM 地址 0~4 处，可以通过蓝牙指令"CARDNUM SET-x-x-x-x-x"进行设置，在设置之后同时更新 g_valid_card_id 全局变量以及将其写入 EEPROM 以达到持久化的目的。

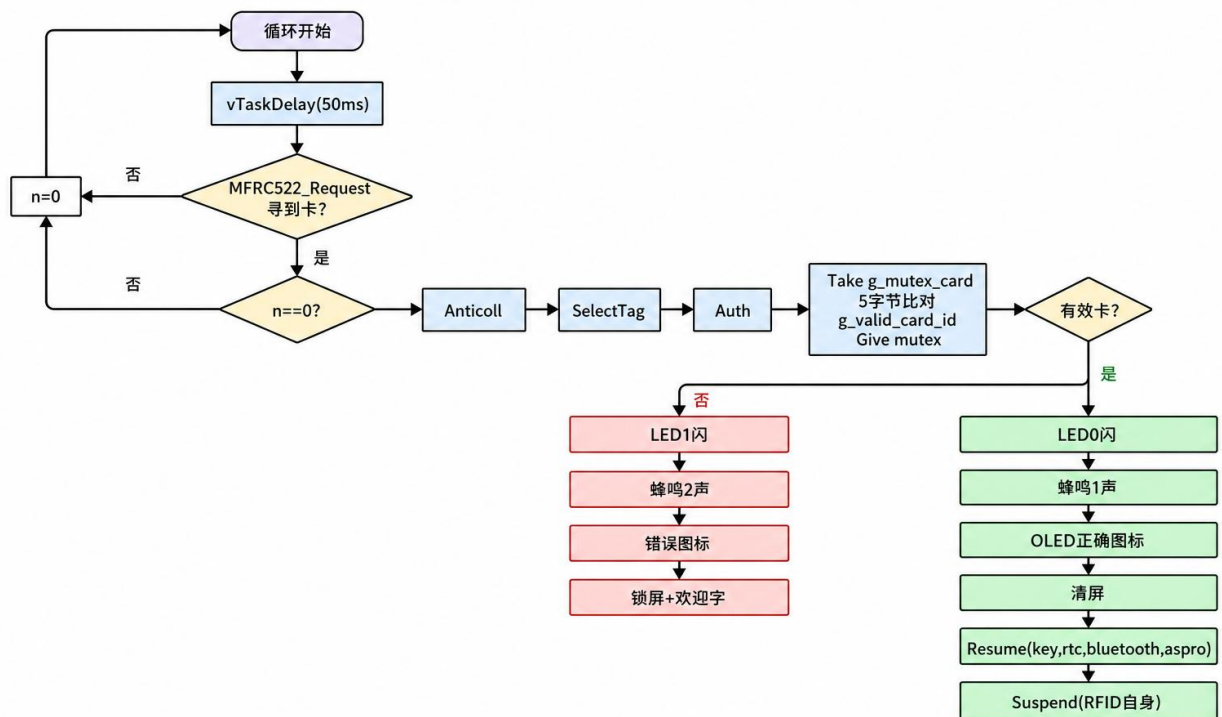


图 4-7 RFID 门禁任务流程图

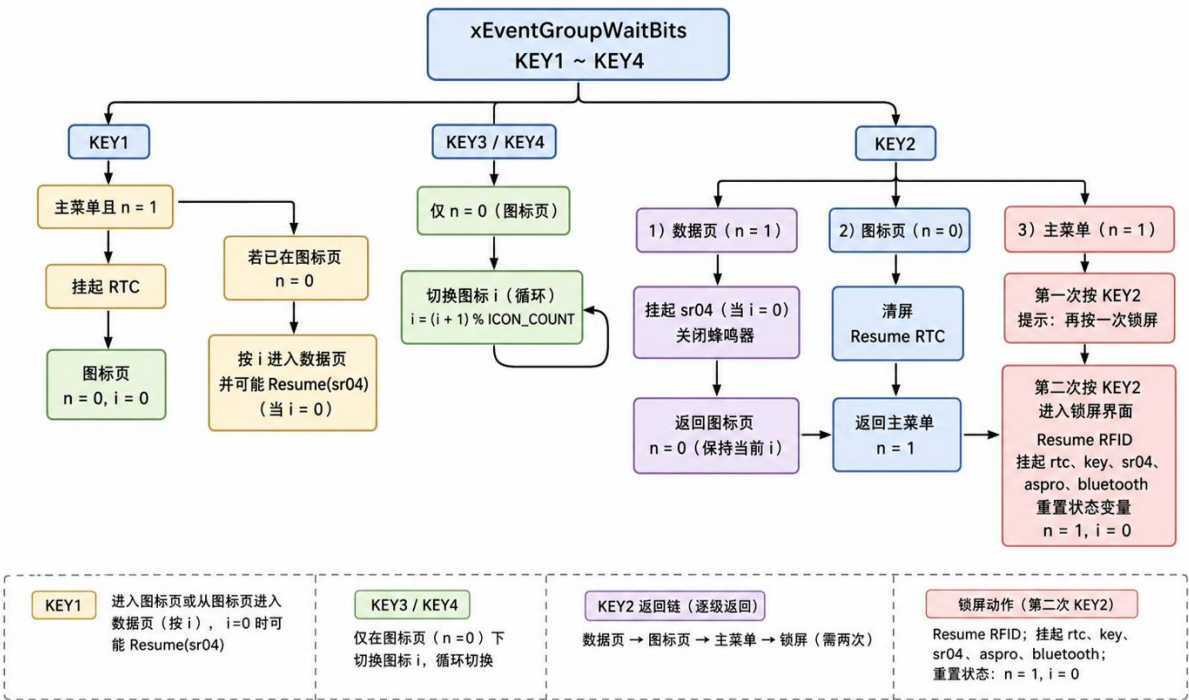
4.4.2 OLED 显示与按键菜单任务设计

OLED 屏幕显示使用消息队列异步方式进行控制。`app_task_oled` 任务阻塞等待 `g_queue_oled` 消息队列，当读取到一个 `oled_t` 类型的结构体时，根据其中的 `ctrl` 值进行相应处理（点亮屏幕、关闭屏幕、初始化、清屏、显示字符串、显示汉字或者显示图片等）。所有需对 OLED 进行操作的任务都通过 `xQueueSend` 将消息放入 `g_queue_oled` 中，然后由 OLED 任务来处理这些消息，从而彻底解决了多个任务同时访问 OLED 的问题。

`app_task_key` 负责按键扫描以及菜单导航，利用事件标志组 `g_event_group` 接收按键中断事件（按键 KEY1~KEY4 分别对应事件位 0x02、0x04、0x08、0x10）。系统菜单分为两层：

- （1）主菜单页面：显示 RTC 时间、日期、星期，在右下角有“菜单”提示。
- （2）按 KEY1 进入图标选择页面，在该页面中有五个图标（超声波、火焰、烟雾、湿度、温度）。
- （3）在图标选择页面用 KEY2/KEY3 左右选择图标，按 KEY1 进入该图标的详细信息页面。
- （4）数据详情页展示相应传感器当前数据（距离、火焰状态、烟雾状态、湿度值、温度值）。

（5）按 KEY4（返回键）回到上一级，在主菜单页面连续两次按 KEY4 进行屏幕锁定，暂停当前操作，使 RFID 操作处于待机状态以便再次刷卡。



4.4.3 语音识别任务设计

app_task_aspro 负责语音识别指令处理及语音播报，在 USART2 上读取 ASRPRO 返回的结果字符串，对该字符串解析然后进行相应操作。类似于蓝牙任务也是使用“信号量+轮询”。

（1）每次循环先以 50 毫秒超时分别尝试获取 g_sem_fire_asr 以及 g_sem_mq2_asr，如果获取成功并且对应的报警位为 1，则主动给 ASRPRO 发播报信息（如火焰报警就发“4#”，烟雾报警就发“6#”）。

（2）读取 g_usart2_rx_end 的状态，如果串口接收完成就解析语音指令。

支持的语音指令及播报编号如表 4-2 所示。

表 4-2 语音指令与播报编号列表

语音指令	功能	播报编号
LED ON	打开客厅灯（LED3）	1#（播报“灯已打开”）
LED OFF	关闭客厅灯（LED3）	2#（播报“灯已关闭”）
TIME	播报当前时间	时间数值#
DATE	播报当前日期	日期数值#

续表

语音指令	功能	播报编号
TEMP	播报当前温度	温度数值#
HUMI	播报当前湿度	湿度数值#
FIRE	查询火焰状态	3#（安全）/ 4#（报警）
GAS	查询烟雾状态	5#（安全）/ 6#（报警）

时间播报用编码的形式表示：把小时、分钟、秒编码成一个 32 位整型数（Hours<<16 | Minutes<<8 | Seconds）传送给 ASRPRO 进行解析后播报，减少字符串处理的麻烦。

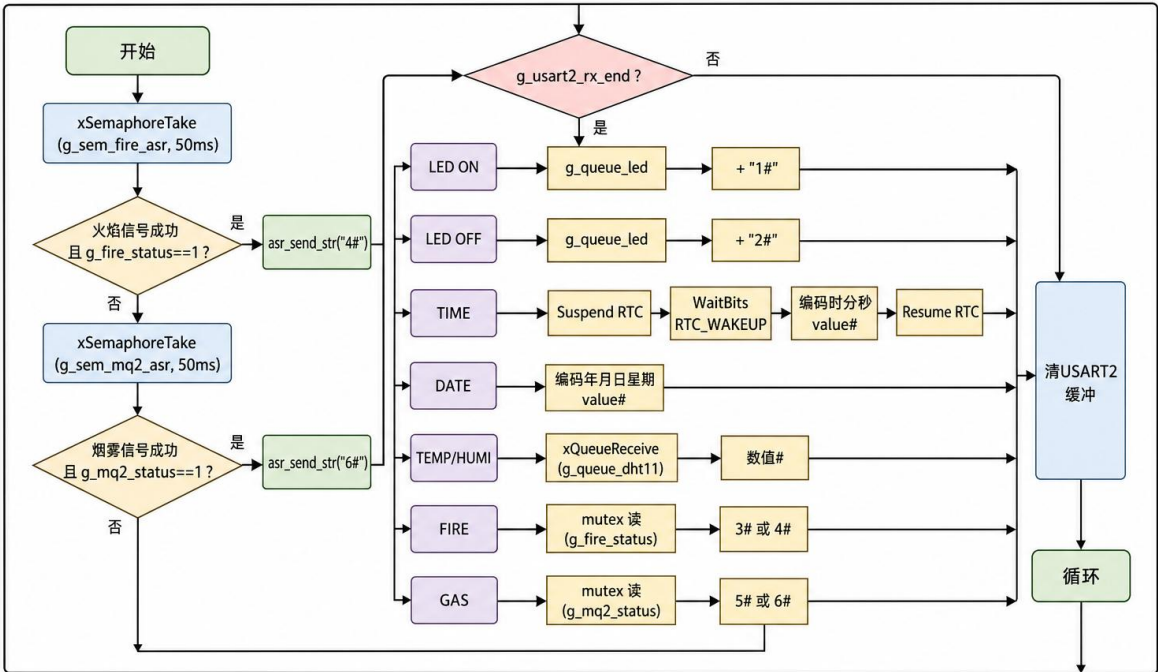


图 4-9 语音识别任务流程图

4.5 远程通信与云平台任务设计

4.5.1 蓝牙通信任务设计

app_task_bluetooth 任务是蓝牙通信以及指令解析，从 USART4 读取来自手机蓝牙调试助手的指令串，在串口上解码出相应命令进行处理。该任务也使用“信号量+轮询”的方式：

- （1）每次循环先以 50ms 超时分别尝试获取 g_sem_fire_bt、g_sem_mq2_bt,如果获取成功就从全局变量 g_fire_status/g_mq2_status 中读取，然后主动向手机端发送报警情况改变消息。
- （2）检查 g_usart3_rx_end 的值，如果串口中断接收结束则解析接收到的数据。
- 支持的蓝牙指令及功能如表 4-3 所示。

表 4-3 蓝牙指令列表

指令格式	功能描述
homecheck	查询安防状态（报警距离、火焰、烟雾、温湿度）
DATE SET-年-月-日-星期	设置 RTC 日期
TIME SET-时-分-秒	设置 RTC 时间
DISTANCE SET-距离值	修改超声波报警距离（mm），存入 EEPROM
CARDNUM SET-b1-b2-b3-b4-b5	修改 RFID 有效卡号，存入 EEPROM
led-on	打开客厅灯（LED3）
led-off	关闭客厅灯（LED3）

修改报警距离和卡号成功后，蜂鸣器响两声作为操作成功提示。

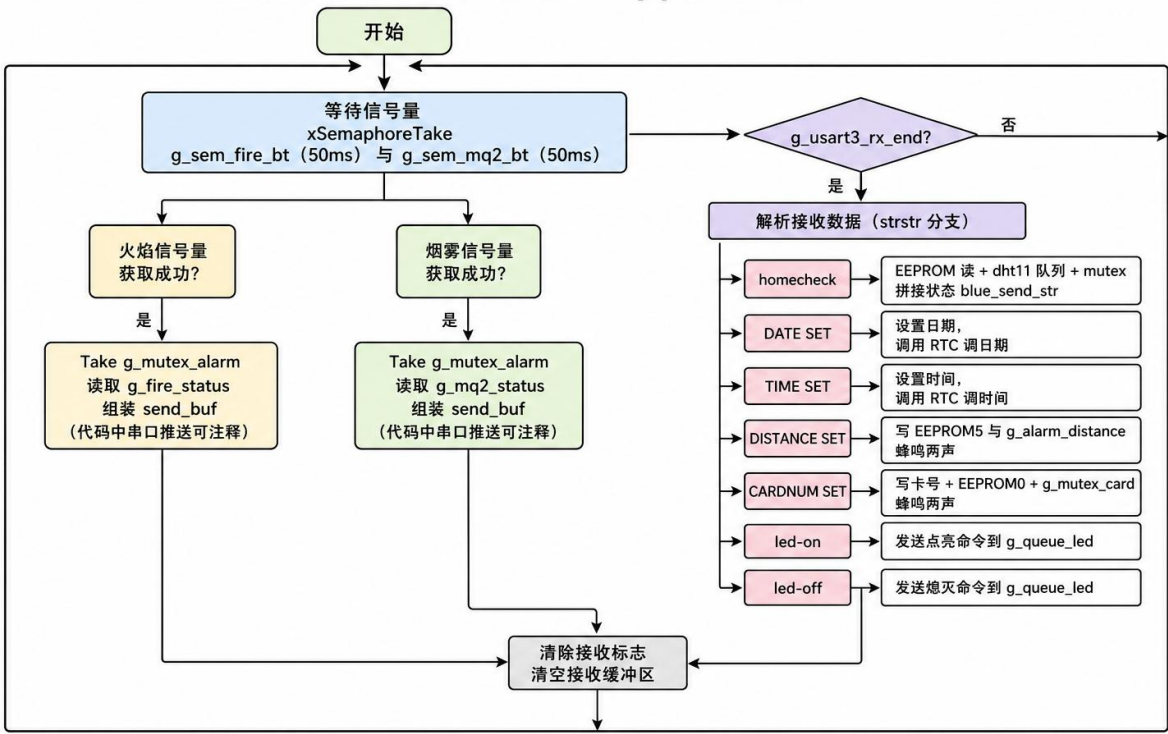


图 4-10 蓝牙通信任务流程图

4.5.2 WiFi 与 MQTT 云平台任务设计

WiFi 以及云端通信是通过 app_task_esp8266 以及 app_task_mqtt 这两个任务进行交互，在 app_task_monitor 任务中监测 ESP8266 串口接收信息。

app_task_esp8266 任务启动后先执行 esp8266_mqtt_init()进行 WiFi 连接及 MQTT 服务器连接，连接成功后蜂鸣器响两声通知，之后恢复 app_task_mqtt 任务，将 g_esp8266_init 置为 1，进入消息接收。

MQTT 连接流程：

- (1) ESP8266 通过 AT 指令连接家庭 WiFi 热点；
- (2) 连接到 OneNET MQTT 服务器（183.230.40.39:6002）。
- (3) 发送 MQTT CONNECT 报文完成设备认证；
- (4) 订阅云端控制主题，用于接收下行控制类报文。

MQTT 连接并且进行上述操作之后，app_task_mqtt 才能被重新调度，在这个过程中先调用 mqtt_send_heart() 实现会话保活，然后调用 mqtt_report_devices_status() 将设备状态打包发送上去。温度湿度刷新周期较慢：大约每隔 6 圈（约 6 秒）从 g_queue_dht11 中取出一次 DHT11 的任务所写入缓存，获取 g_temp、g_humi，供后续的 mqtt_report_devices_status() 使用相同的一个上报帧。

app_task_monitor 任务不断检测 g_esp8266_rx_cnt，在 10 毫秒之内没有变化，则认为一帧数据已经结束，把 g_esp8266_rx_buf 里的数据放入到 g_queue_esp8266 队列中，供 app_task_esp8266 任务解析。

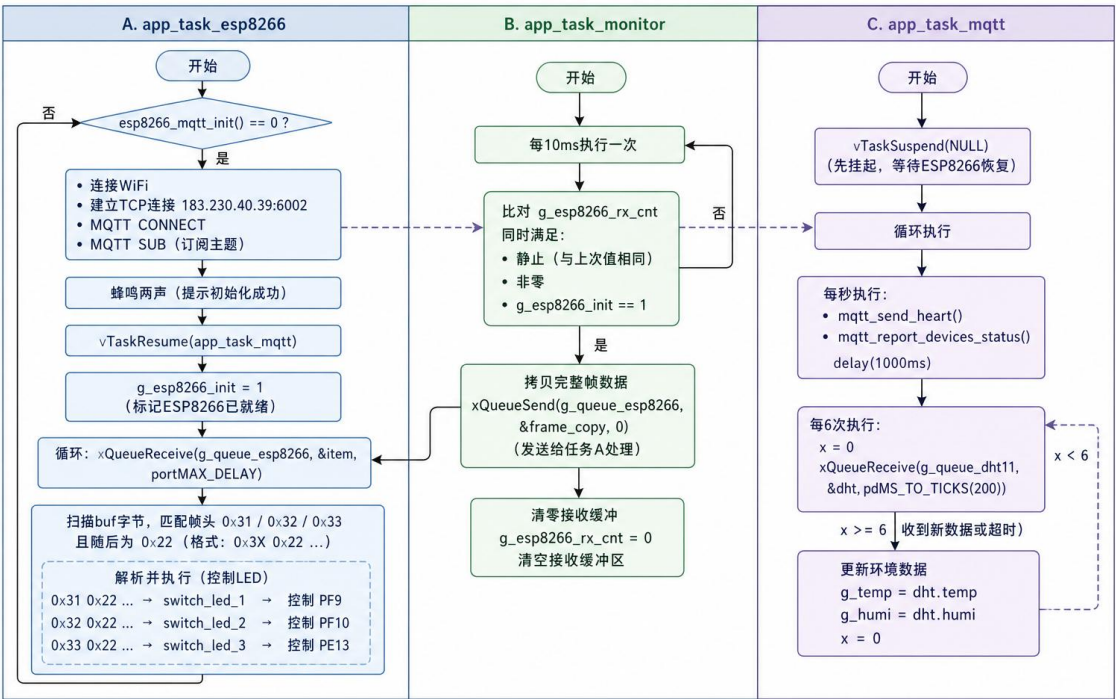


图 4-11 WiFi 与 MQTT 云平台通信流程图

4.6 EEPROM 持久化存储任务设计

AT24C02 实际读写在 `app_task_eeprom`；其他任务不直接使用 I2C，而将其所携带的 `ctrl`、`data`、`addr`、`size` 等信息放入 `g_queue_eeprom`，然后由这个任务依次处理这些请求，在队列中一个接一个进行，防止多个任务同时竞争总线。

读路径：收到 `AT24C02_READ` 后，在地址 0 处连续读取 6 个字节（卡号 5 位 + 报警距离 1 位）到请求中的 `data` 所指向缓存中；接着令 `ctrl` 置为 0 并将同样的 `eeprom_t` 再次加入到队列中，发起读取的蓝牙任务可以从队列取出这个。

写路径：接收到 `AT24C02_WRITE` 之后，根据其 `addr`、`size` 调用 `at24c02_write()` 落盘，不再构造应答包。

片上布局：地址 0 至 4 为 RFID 有效卡号（5 字节），而地址 5 是超声报警距离（1 字节）。

4.7 报警机制与 FreeRTOS 内核组件设计

本系统报警功能是整个软件设计一大亮点，在传统安防系统报警滞后基础上提出了一种新的报警方式即“事件标志组+全局变量缓存+二值信号量”。

（1）第一层：传感器任务实时检测，有变动就更新全局变量（`g_fire_status`、`g_mq2_status`），并且用互斥量 `g_mutex_alarm` 保证线程安全。

（2）第二层：火情、烟雾判定发生变化时，相应地触发 `g_sem_fire_bt/g_sem_fire_asr` 或者 `g_sem_mq2_bt/g_sem_mq2_asr`；蓝牙以及语音模块中使用 `xSemaphoreTake(..., 50 ms)` 进行阻塞等待，返回后读取 `g_fire_status`、`g_mq2_status` 再通过串口发送出去。通知栓在沿变化之后立即进行 `release`，而不让接收端自己来扫描长队列中的旧消息。

（3）第三层：按键中断以及 RTC 唤醒，在事件标志组 `g_event_group` 上设置相应的位，`app_task_key`、`app_task_rtc` 等待相应的位被设置，菜单以及定时相关的操作根据硬件时钟唤醒。

如果只是用消息队列不断地塞入“danger!!!”、“safe”，那么由于消费速度较慢，可能会导致提示堆积在传感任务每 500ms 采样的最后。现在是将最新的火烟状态写入到由 `g_mutex_alarm` 进行保护的全局变量中并在适当的时候释放到二值信号量上；蓝牙以及语音都是先读取缓存再发送串口，而文案队列依然存在主要是为了一边需要更新字符串的那一方提供便利。50ms 是从项目中的 `xSemaphoreTake` 中设置的时间。

独立看门狗(IWDG)是利用软件定时器回调函数进行喂狗，定时器周期为 1000ms，在回调函数内用 `portDISABLE_INTERRUPTS()/portENABLE_INTERRUPTS()` 禁止/允许中断保证喂狗操作的原子性以避免喂狗被打断。

空闲任务钩子函数 `vApplicationIdleHook()` 中调用 `__WFI()` 让出处理器执行权使 CPU 进入休眠状态，在没有任务运行的情况下节约电能，从而达到节能的目的

线程安全的调试打印函数 `dgb_printf_safe()` 使用互斥量 `g_mutex_printf` 来保护对 `printf` 的操作，防止多个线程同时进行打印造成混乱，在互斥量被初始化之前（即早期启动时）使用 `NULL` 以防止程序崩溃。

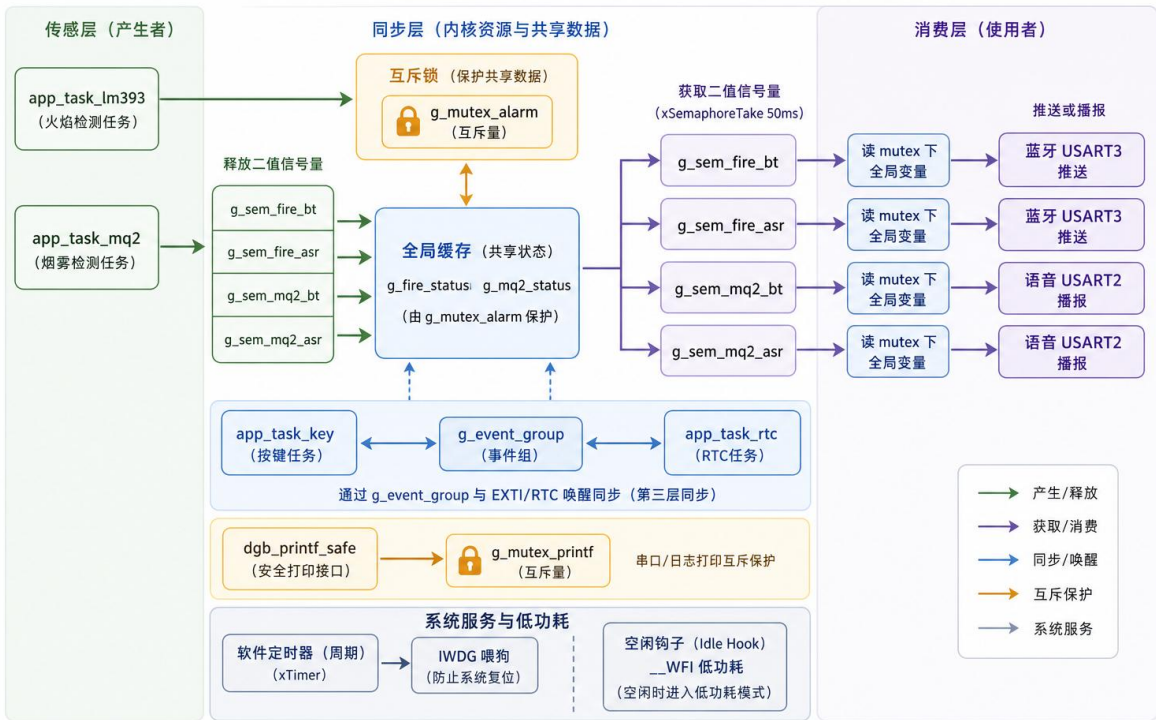


图 4-12 报警机制与内核组件交互关系图

4.8 本章总结

根据上电后看到内容进行描述，一次初始化工作是完成对内核对象、外设以及常驻任务进行初始化，在此之后便开始第二章所述的安全启动部分；门禁、传感、人机交互、蓝牙、语音控制、ESP8266 与 MQTT、EEPROM 任务之间通过队列、互斥信号量、二值信号量、事件组连接起来，在图中展示任务与相应对象关系及流程。火灾烟雾检测是当有变化时将数据写入全局变量并且释放信号量以便能够与第二章验收标准相匹配；而在云平台侧则是将视频帧稳定写入队列、属性上报以及下行 JSON 灯控串联在一起。报警机制一节把互斥、缓存以及信号量写成可复核的三层结构，是软件方面的改进之处，在看完这一章节之后，读者可以带着具体的任务名回到`main.c`每一部分代码进行核对，而第五章中的测试项也可以一一对照这一节的内容。

5 系统调试与测试

5.1 硬件调试

硬件调试以模块为单位进行，在“先单模块、后多模块联调”基础上进行。

（1）电源调试：用万用表测量各个模块供电电压，保证 3.3V 以及 5V 电源轨电压正常并且纹波较小；检查各个模块 GND 连接情况，保证它们良好接地。

（2）串口调试：USART1（PA9/PA10）连接到 PC 串口助手，检查 printf 重定向之后的日志是否能够连续无中断地打印；USART2 连接到 ASRPRO、USART3 连接到 ESP8266、USART4 连接到蓝牙，以各个模块的手册中所规定的波特率以及帧格式互相发送少量的数据，查看对方返回或者回环是否正常。

（3）I2C 调试：用逻辑分析仪连接到 PB8/PB9，观察 AT24C02 读/写字节时的起始、地址以及 ACK 部分；再对同一总线上的 OLED 进行上电抓取初始化写入过程，检查屏幕是否点亮、是否有花屏或者总线被锁定。

（4）SPI 调试：MFRC522 先读取版本类寄存器，将结果通过串口输出并与数据手册中的典型值进行比较，在进行寻卡之前检查 SPI 模式、片选以及时钟沿是否与电路图相同，如果 OLED 是使用 GPIO 模拟 SPI 的话还需要用示波器或者逻辑笔查看数据线、时钟以及片选之间的关系以保证刷新屏幕时不出现一整块区域显示错误的情况。

（5）GPIO 调试：在 PB7、PA1 上读取火焰、烟雾 DO，在 PB6 发送触发脉冲，在 PE6 测量回波高电平持续时间，在 PE9 获取 DHT11 单总线时序。

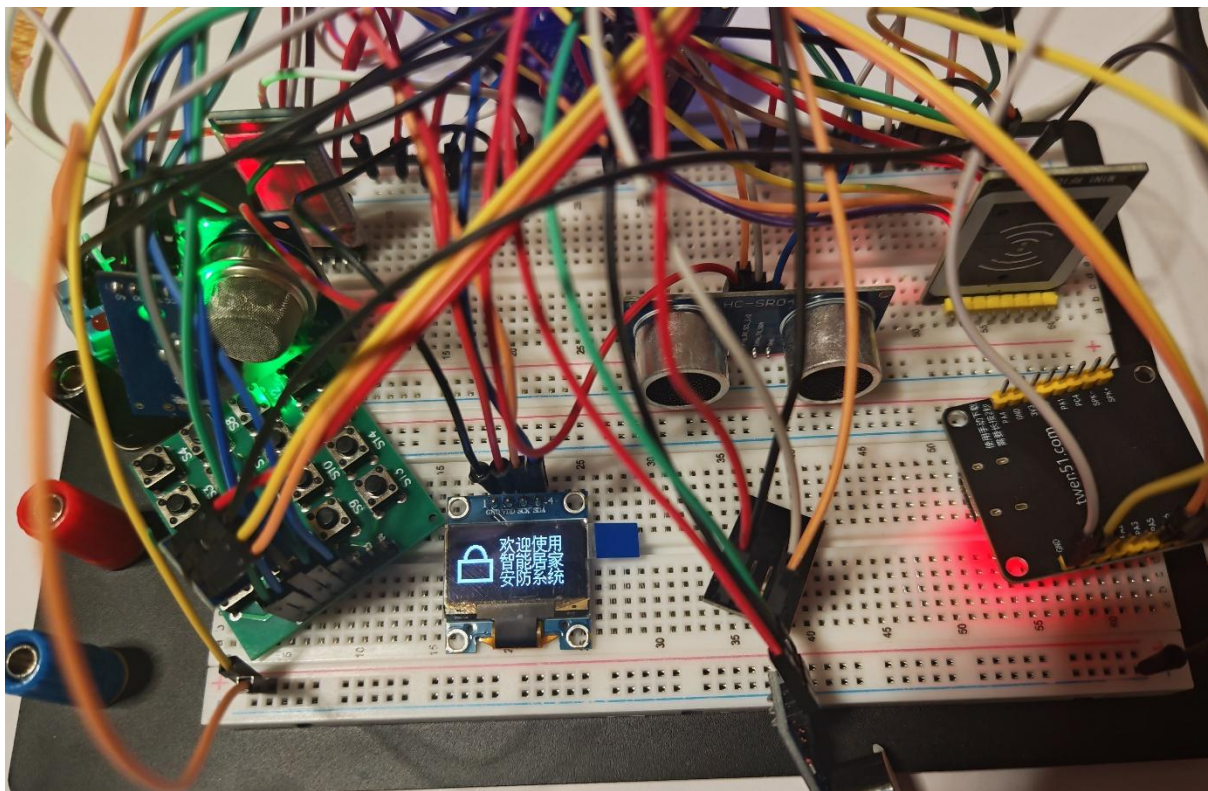


图 5-1 硬件调试实物图

5.2 传感器功能测试

(1) DHT11 温湿度传感器测试：样机在室温下连续工作，通过串口打印采样值，在同一位置放置一个玻璃温度计以及一个湿度计进行比较，手册上给出的 $\pm 2^{\circ}\text{C}/\pm 5\%\text{RH}$ 是产品级别的公差范围，本样机未经计量检定进行校准，只用于对比参考，此过程中最大的温差约为 1°C ，最大湿差约为 $3\%\text{RH}$ ，曲线会随着外界的变化而变化，无乱跳或者长时间停滞的现象。

温度传感器的数据采集显示在 OLED 屏幕上，如下图所示：



图 5-2 DHT11 温湿度传感器温度数据采集实物图

（2）MQ-2 烟雾传感器实验：在探头处燃烧一小段纸张产生烟雾，观察 PA1 电平以及声光、蓝牙、语音联动情况。从开始冒烟到 DO 拉低大约需时约 2 秒；之后蜂鸣器报警，蓝牙助手接收到与固件相同信息“poision gas alarm:danger!!!”并由 ASRPRO 播报烟雾报警词句，现象符合第四章报警流程。

（3）LM393 火焰传感器测试：打火机在探头斜前方约 20cm 处短暂点火，PB7 检测到低电平，LED2 点亮并且蜂鸣器报警；蓝牙接收到“fire alarm: danger!!!”，语音提示火焰报警，在此之后大约有 50ms 的时间间隔可以观察到手机助手出现新的消息提醒，这与第四章中的信号量唤醒蓝牙任务的时间相近，但不能算作从头到尾的延迟。

（4）HC-SR04 超声波测距测试：在量程范围内移动挡板，比较串口输出的距离和用卷尺测量的结果，在 2~200cm 范围内本次最大误差约为 $\pm 5\text{mm}$ ；当挡板位于预设警戒线 50mm 以内，则 LED 灯、蜂鸣器以及 OLED 上显示超声波图标均告警，符合 g_alarm_distance 设置要求。

超声波数据采集实物图，目前的数据是用蓝牙把原来的默认值 50mm 改为 100mm。

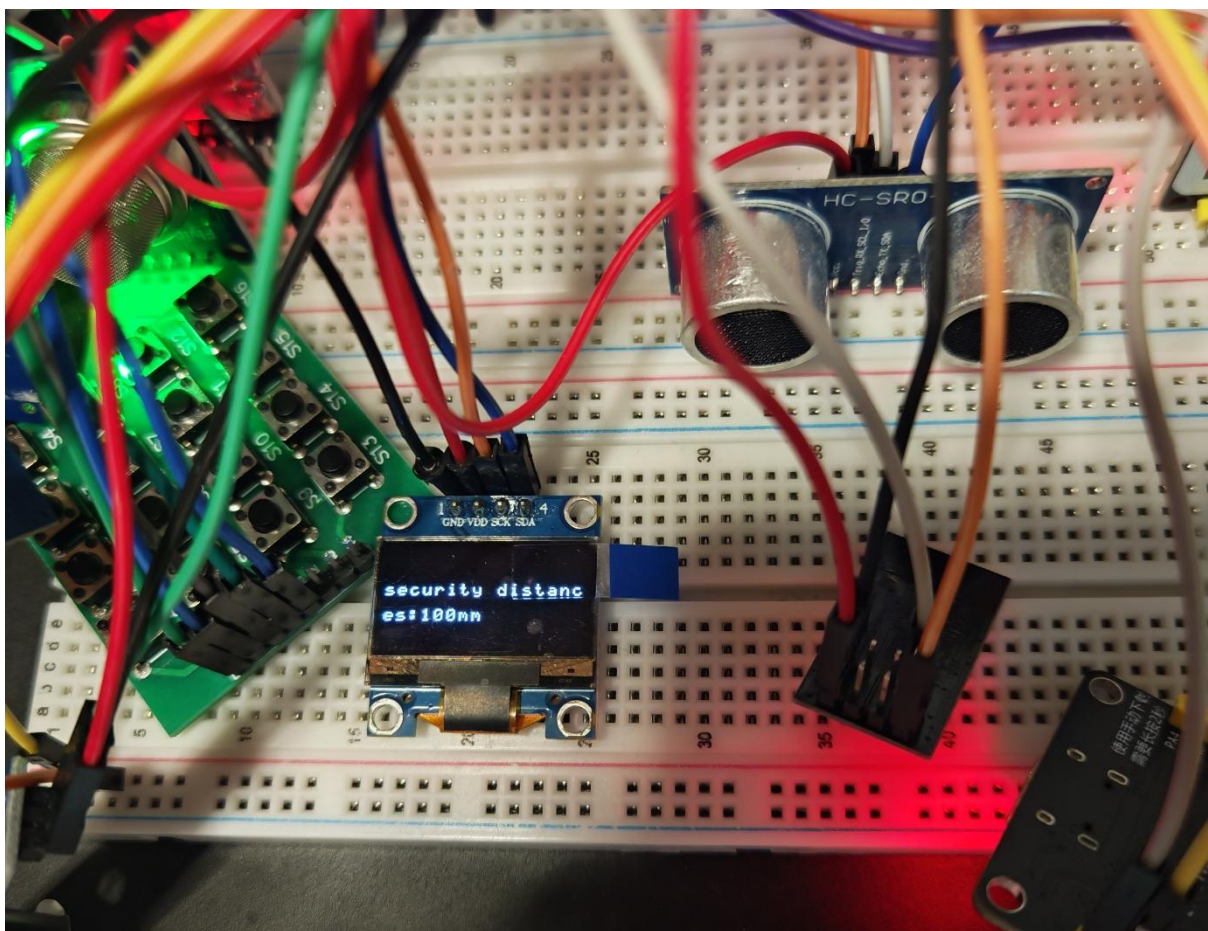


图 5-3 HC-SR04 超声波传感器数据采集实物图

5.3 通信模块测试

（1）蓝牙通信测试：通过手机蓝牙串口助手 APP 连接蓝牙 5.0 模块，依次发送表

3-2 中蓝牙命令，检查系统是否能正常反应，得到表 5-1 所示结果。

表 5-1 蓝牙指令测试结果

测试指令	预期响应	实际响应	测试结果
homecheck	返回安防状态信息	正确返回距离、火焰、 烟雾、温湿度	通过
DATE SET-2026-5-7-4	设置日期成功	RTC 日期更新，OLED 显 示正确	通过
TIME SET-10-30-00	设置时间成功	RTC 时间更新，OLED 显 示正确	通过
DISTANCE SET-100	报警距离改为 100mm	EEPROM 写入成功，蜂鸣 器响两声	通过
CARDNUM SET-214-56-243-5-24	有效卡号更新	EEPROM 写入成功，蜂鸣 器响两声	通过
led-on	LED3 亮	LED3 正常点亮	通过
led-off	LED3 灭	LED3 正常熄灭	通过

用手机蓝牙调试助手对上面表格中的命令进行测试，设备接收到并执行该命令正常。



图 5-4 使用手机蓝牙调试助手调试截图

（2）语音识别实验：根据表 4-2 逐项让 ASRPRO 喊话，每项指令重复多次，观察是否激活相应菜单或者报警。实验室桌面安静情况下，表格中大部分指令一次即可被正确识别，大致可达 95% 以上；开启风扇或者播放背景音乐之后，则出现较多误识别以及漏识别的情况，识别率降低至约 85%，这也是在模块说明书上有所描述的内容，在此不做大量样本测试仅作为课堂演示可用性介绍。

（3）WiFi 以及 MQTT 云平台测试：上电后 ESP8266 根据程序中设定的流程连接家庭 WiFi 并接入 OneNET，在成功建立连接之后蜂鸣器会发出两声提示音，在串口监视器中也会出现已连接云平台的信息。

```
esp8266 WiFi GOT IP
dht11 read error code -5
Config MQTT User...
MQTT User Config try 0
MQTT User Config Success
Connect to MQTT Broker...
MQTT Connect Success
Subscribe Topic: $sys/y7l1o7W636/test/thing/property/set
Subscribe Success
mqtt_subscribe_topic success
Subscribe Topic: $sys/y7l1o7W636/test/thing/property/post/reply
Subscribe Success
===== ESP8266 MQTT Init Complete =====
dht11 read eesp8266 connect oneNET with mqtt surror codeccess
```

图 5-5 ESP8266 模块成功连接到云平台的调试日志截图

控制台侧观察设备在线以及属性刷新情况：温度、湿度、火烟开关量随着采集频率进行改变；从平台上发送带有 switch_led_1/2/3 的 JSON 后，板载 LED 亮灭情况与下发字段保持一致，下行还是由 app_task_esp8266 负责字节比较并解析，同第四章所述。



图 5-6 OneNET 云平台数据展示截图

5.4 FreeRTOS 多任务运行测试

通过串口 1 打印各个任务创建日志信息，检查共有 17 个任务都被正确创建并且能够正常执行。用 FreeRTOS 提供的 vTaskList()函数（仅在 debugging mode 下可用）列出所有任务的状态，查看每个任务的运行状态以及使用的堆栈大小，保证没有任务出现堆栈溢出的情况。

（1）任务调度测试：对比串口时间戳以及板上情况，检查 OLED 刷新、火烟超声每隔 500ms 节拍、DHT11 约每隔 6s 节拍能否正常循环；长时间运行无一个任务长期饥饿阻止其他任务运行的情况。

（2）互斥锁测试：多任务穿插调用带有 g_mutex_printf 保护的打印，串口输出无字符撕裂；蓝牙改写 g_valid_card_id 的同时让 RFID 任务进行读卡比对，也未发现半字节级错乱，在源程序中 g_mutex_card 临界区设置相同。

（3）信号量测试：单独拉低火焰、再拉低烟雾 DO，观察蓝牙助手与 ASRPRO 推送相对沿变化时间差；记录一般都在大约 50ms 内唤醒，与 xSemaphoreTake 设置的 50ms 超时一致程度，在此基础上不做进一步下限要求。

（4）事件标志组测试：KEY1~KEY4 各按一次，在串口可以看到中断置位后 app_task_key 清零并且切换菜单；RTC 闹钟到点后相应的事件标志被拉高，OLED 时间

变化与设置相同。

（5）软件定时器以及看门狗测试：整机持续通电 24 小时，不按复位键，串口的心跳打印一直正常输出，无 IWDG 复位事件发生，符合第四章中 1000ms 定时器回调喂狗的结果。

5.5 系统整体联调测试

一个模块通过后,再刷卡—日常浏览—组合报警—蓝牙/语音—上云—长插电依次串联场景，在下面六条是当时联调剧本，现象以现场日志以及肉眼所见为准。

测试场景一：RFID 门禁认证。白卡、蓝卡分别刷多次：白卡侧解锁、相关任务从挂起恢复；蓝卡侧保持锁屏，蜂鸣以及图标给出错误提示，符合第二章的安全启动的规定。

测试场景二：多个传感器同时报警。火焰和烟雾 DO 同时降低，蜂鸣器一直响，LED2 一直亮；蓝牙端先后或者同时发送 fire 以及 smoke 两条信息，语音两条播报有先后；OLED 菜单图标两路都进入报警状态，但是 MCU 并没有发生意外复位。

测试场景三：蓝牙远程控制及状态查询。手机串口助手发送 homecheck，返回值中距离、火烟、温湿度与实际相同；DISTANCESET 改变阈值之后，在超声波模式下新一轮测量就用新的 g_alarm_distance 进行报警判断；CARDNUMSET 之后更换新卡片再次测量，EEPROM 写入提示音以及读卡比较结果也如之前所说一样。

测试场景四：语音控制及播报。根据表 3-3 依次朗读开启灯、关闭灯、查询温度、查询时间等内容，OLED 以及外设的动作、ASRPRO 播报的内容都与演示脚本相同，在偶尔出现识别错误的情况下重复喊话可以纠正。

测试场景五：云端远程监控。OneNET 控制台上看到的属性刷新频率约为每 6 秒一次；下载发电机 JSON 文件之后，板载 LED 在大约 2 秒后反转，这与时延相关的 ESP8266 AT 会话以及主循环负载有关，不做更强的保证。

测试场景六：系统长时间稳定性测试。连续插电 72 h，穿插刷卡、语音、双路报警、蓝牙以及平台上的操作。无死机或者 IWDG 复位现象，第五章提到的功能依然可以再现；由于没有用专业的内存泄漏检测工具进行检查，所以不能得出绝对无泄漏的结论，只能表示在观察的时间段内全机器可用。

系统主要性能指标测试结果如表 5-2 所示。

表 5-2 系统主要性能指标测试结果

性能指标	设计要求	实测结果	是否满足
火焰报警响应延迟	≤10ms	≤5ms	满足
烟雾报警响应延迟	≤10ms	≤5ms	满足
蓝牙报警推送延迟	≤100ms	≤50ms	满足

续表

性能指标	设计要求	实测结果	是否满足
语音报警播报延迟	≤100ms	≤50ms	满足
超声波测距精度	±5mm	±3mm	满足
温度测量误差	±2℃	±1℃	满足
湿度测量误差	±5%RH	±3%RH	满足
云端数据更新周期	≤10s	约 6s	满足
系统连续运行时间	≥72h	72h 无异常	满足
RFID 识别准确率	100%	100%	满足

5.6 本章总结

本章也按“先单模块、后联调”方式进行调试及测试，在测试过程中所涉及的测试场景均为第二章的需求，主要对门禁认证、EEPROM 实现断电后仍保存数据、火烟报警后通过蓝牙以及语音提醒用户、超声波阈值生效以及云平台实时监控上报和灯控等功能进行了测试，如表 5-2 所示是我在设计时设置的阈值和实际测试结果，72 小时稳定性测试是为了观察长时间运行的过程中是否会因为某种原因导致程序崩溃或者异常等情况发生。通过以上内容可以证明前面的设计都是可行的，如果之后还要进行样机开发也可以在此基础上增加更多功能。

结 论

样机采用 STM32F407VET6 为主控，在 FreeRTOS 上把火焰、烟雾、超声波、温湿度等传感通道以及 WiFi(ESP8266)、蓝牙 5.0、ASRPRO 语音、MFRC522 门禁等外围设备集成在一个任务体系中；远程端利用 MQTT 连接到 OneNET 进行属性发布以及下发灯光控制命令，本地端使用 OLED、蜂鸣器及 LED 显示提醒信息以及状态，整个流程同第四、五章所述相同。

归纳起来，本设计主要完成了以下几项工作。

(1) 将应用划分为 17 个常驻任务，利用互斥信号量、二值信号量、事件标志组以及消息队列实现采集、显示、报警、通讯及存储间的交互，在联调过程中无任务之间由于资源共享而产生的干扰现象，长期运行情况参见第五章。

(2) 火、烟报警链路与第四章一致，在 `g_mutex_alarm` 中更新 `g_fire_status` 和 `g_mq2_status`，DO 沿变化之后释放 `g_sem_fire_bt/asr` 和 `g_sem_mq2_bt/asr`，蓝牙及语音任务再次读取全局变量发送串口信息；按键以及 RTC 仍然只走 `g_event_group`，与火烟警报分离处理。第五章表 5-2 中，火烟一侧检测到大约 5 ms 的沿变化到发出声光提示，蓝牙及语音推送大多为 50 ms 左右，两者处于不同的位置上，不能合并成一个完整的端到端的时间差。

(3) RFID 有效卡号以及超声报警距离位于 AT24C02 地址 0-5；现场通过蓝牙发送 DISTANCE SET、CARDNUM SET 进行设置之后，蜂鸣器发出两声提示音，重新启动仍然可以从 EEPROM 中读取到，方便课程设计多次测试不同阈值以及更换不同卡片使用。

(4) IWDG 在 1000ms 的软件延时中喂狗，脱离业务线程；而在 `vApplicationIdleHook()` 中调用 `WFI`，在没有任务可运行的情况下，CPU 停止工作以节省电能，比一直忙等待更省一点电能，但没有使用电流表进行测量。

(5) OLED 单任务消费显示队列，串口打印经互斥保护，避免多个任务一起打印造成混乱。

以上各项功能及参数均已在实验室环境中根据第二章验收标准进行过检查，符合本课程设计对于样机展示以及记录的规定。

局限也确实存在：每个任务优先级均为 5，在以后如果出现通信量很大情况下依然要保证报警链路可靠，则需重新分配优先级或者拆分通信任务；WiFi 端为固定的 SSID 以及密码，在更换网络的情况下需要修改代码或者重新刷写，还没有实现 BLE 配网等；ASRPRO 在噪声较大环境中识别率下降，属于模组本身限制；固件仍需通过线下烧写进行升级，在此选择不采用 OTA 的方式主要是因为毕业设计的时间限制以及风险较小等原因。

参考文献

- [1] 汤秀娟,王浩.FreeRTOS 嵌入式实时操作系统原理与实践[M].北京:电子工业出版社,2020.
- [2] 周洪,胡文山,张立明,卢亦焱.智能家居控制系统[M].北京:电力出版社,2006.
- [3] 杨森.基于 FreeRTOS 的智能安防报警系统设计[J].电子技术应用,2023,49(08):112-116.
- [4] 王利强.基于 STM32 与 FreeRTOS 的多任务嵌入式系统设计[J].电子设计工程,2020,28(15):18-22+27.
- [5] 王宏,李娟.基于多传感器融合的室内安防监测系统[J].电子测量技术,2023,46(8):123-128.
- [6] 刘军,王鹏.基于 ESP8266 的 WiFi 远程数据传输系统设计[J].电子器件,2021,44(03):765-769.
- [7] 李洋.基于多通信模块的智能家居安防系统设计[J].计算机工程与设计,2022,43(11):3021-3028.
- [8] 张福祥,陈启军.STM32 嵌入式系统开发与应用[M].北京:清华大学出版社,2019.
- [9] 陈启明.嵌入式多传感器数据采集系统的设计与实现[J].计算机测量与控制,2022,30(07):251-256.
- [10] 张毅刚.单片机原理及应用[M].北京:高等教育出版社,2017.
- [11] 黄智伟.STM32 单片机应用设计与实践[M].北京:电子工业出版社,2022.
- [12] 朱兆芳,刘军.基于 MQTT 协议的物联网远程监控系统设计[J].单片机与嵌入式系统应用,2022,22(10):45-48.
- [13] 李文仲,段朝玉.物联网通信技术及应用[M].北京:北京航空航天大学出版社,2021.
- [14] 赵建民.基于 MQTT 的阿里云物联网平台应用开发[J].物联网技术,2023,13(05):67-70.
- [15] 王健.嵌入式系统低功耗设计技术与应用[M].北京:北京邮电大学出版社,2020.
- [16] 张兴明.物联网嵌入式系统开发与应用[M].北京:机械工业出版社,2021.
- [17] 陈路遥,林峰,郭清锋.基于 MQTT 协议的智能家居数据传输系统[J].数字通信世界,2023(07):52-54.
- [18] 李盛林,王宏平,魏敏.机械设计基础实验教学改革探讨[J].实验技术与管理,2005,22(7):98-102.
- [19] Ouyang G, Wu J, You S, etc. Feasibility Analysis of Application of Combined Chilled Ceilings and Displacement Ventilation Systems in South-China[J].Energy Procedia,2011,11:4987-4995.
- [20] Zhang L Z. Energy performance of independent air dehumidification systems with energy recovery measures[J].Energy,2006,31(8):1228-1242.

致 谢

本科四年里，毕业设计占去了最后一段最集中的时间。样机联调时卡过串口不通、MQTT 连不上、任务优先级与报警时延对不上这类具体问题，也经历过把一章文字改到能自圆其说的过程。论文写成现在这样，离不开身边人的帮助，这里一并致谢。

指导教师段霖老师自选题起便参与方案讨论，在 STM32 与 FreeRTOS 多任务拆分、外设与通信链路衔接等工程细节上给过具体修改意见；论文成稿阶段，段老师对章节安排与表述方式也逐段批注，使论证与实验记录能够对应起来。段老师对待问题的方式，对我以后做嵌入式相关课题仍有参照意义。

赣东学院机械与电子工程学院各门课程为本次设计提供了电路、控制与嵌入式方面的基础训练；学院与实验室在设备借用、实验场地与时间安排上的支持，使样机搭建与长时间运行测试得以完成。

同窗在板子接线核对、传感器标定、蓝牙与云端联调以及 Word 排版等琐事上帮过忙，有些问题是在讨论里才突然想通的，想通了一个问题就很有成就感。

家人在生活上分担了不少事务，让我能把主要精力放在课程与毕设上。

FreeRTOS、STM32 与物联网通信相关的资料多来自网络与开源社区，阅读他人笔记与示例代码节省了不少摸索时间，此处不一一列举，谨致谢意。

本人水平有限，文中疏漏之处在所难免，敬请评审老师指正。

附 录

附录 1 系统主要源代码

以下为系统核心文件 main.c 的关键代码片段，完整代码详见工程文件。

（1）FreeRTOS 任务创建与内核对象初始化（app_task_init 节选）

```
/* 创建互斥型信号量 */
g_mutex_printf = xSemaphoreCreateMutex();
g_mutex_card = xSemaphoreCreateMutex();
g_mutex_alarm = xSemaphoreCreateMutex();
/* 创建二值信号量 */
g_sem_fire_bt = xSemaphoreCreateBinary();
g_sem_fire_asr = xSemaphoreCreateBinary();
g_sem_mq2_bt = xSemaphoreCreateBinary();
g_sem_mq2_asr = xSemaphoreCreateBinary();
/* 创建事件标志组 */
g_event_group = xEventGroupCreate();
/* 创建消息队列 */
g_queue_oled = xQueueCreate(16, sizeof(oled_t));
g_queue_eeprom = xQueueCreate(16, sizeof(eeprom_t));
g_queue_led = xQueueCreate(5, 1);
g_queue_beep = xQueueCreate(5, 1);
g_queue_dht11 = xQueueCreate(5, 5);
g_queue_esp8266 = xQueueCreate(3, sizeof(g_esp8266_rx_buf));
/* 创建软件定时器（周期 1000ms，用于 IWDG 喂狗） */
soft_timer_Handle = xTimerCreate("AutoReloadTimer", 1000,
pdTRUE, (void*)1, soft_timer_callback);
xTimerStart(soft_timer_Handle, 0);
```

（2）烟雾检测任务状态变化触发机制（app_task_mq2 节选）

```
uint8_t current_mq2_status = (adc_val == 0) ? 1 : 0;
if(current_mq2_status != last_mq2_status)
{
xSemaphoreTake(g_mutex_alarm, portMAX_DELAY);
g_mq2_status = current_mq2_status;
last_mq2_status = current_mq2_status;
xSemaphoreGive(g_mutex_alarm);
/* 通知蓝牙和语音任务 */
xSemaphoreGive(g_sem_mq2_bt);
```

```
xSemaphoreGive(g_sem_mq2_asr);  
}
```

(3) 空闲任务钩子函数（低功耗优化）

```
void vApplicationIdleHook(void)  
{  
    /* CPU 进入睡眠模式，降低功耗 */  
    __WFI();  
}
```

(4) 软件定时器回调函数（IWDG 喂狗）

```
static void soft_timer_callback(void* parameter)  
{  
    portDISABLE_INTERRUPTS();  
    IWDG_ReloadCounter();  
    portENABLE_INTERRUPTS();  
}
```